

31.2. Notación adicional de los paquetes en UML

Por último, a propósito de los paquetes, UML proporciona una notación alternativa para ilustrar paquetes internos y externos. Algunas veces es difícil dibujar una caja de paquete externo alrededor de paquetes internos. Las alternativas se muestran en la Figura 31.6.

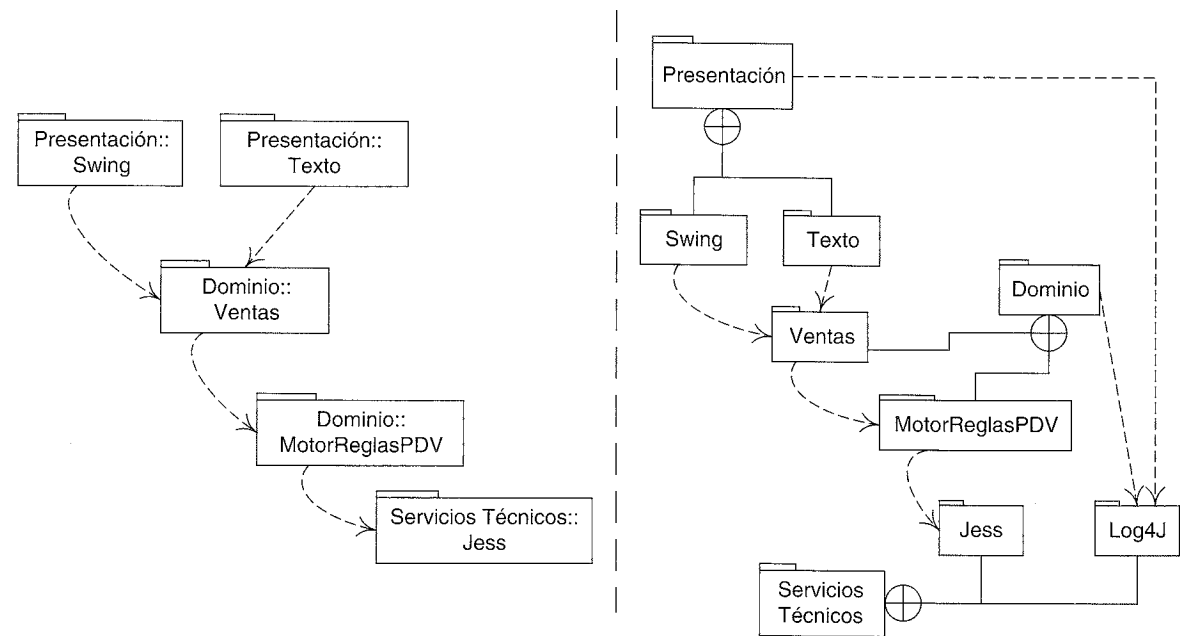


Figura 31.6. Enfoques UML alternativos para mostrar la estructura de paquetes, utilizando nombres de camino UML, o el símbolo de un círculo con una cruz.

31.3. Lecturas adicionales

La mayoría del trabajo detallado —no es sorprendente— sobre las mejoras en el diseño de paquetes para reducir el impacto de las dependencias procede de la comunidad de C++, aunque los principios se pueden aplicar a otros lenguajes. *Designing Object-Oriented C++ Applications Using the Booch Method* de Martin [Martin95] cubre bien el tema, al igual que *Large Scale C++ Software Design* [Lakos96]. El tema también se introduce en *Java 2 Performance and Idiom Guide* [GL99].

INTRODUCCIÓN AL ANÁLISIS ARQUITECTURAL Y EL SAD

Error, no hay teclado – presione F1 para continuar.
mensaje inicial de la BIOS del PC

Objetivos

- Crear tablas de factores de la arquitectura.
- Crear memorándums técnicos que recojan las decisiones acerca de la arquitectura.
- Conocer los principios básicos del diseño arquitectural.
- Conocer recursos para aprender los patrones de arquitectura.

Introducción

La esencia del análisis arquitectural es identificar los factores que deberían influir en la arquitectura, entender su variabilidad y prioridad, y resolverlos. La parte difícil es conocer qué hay que preguntar, valorando los compromisos y conociendo las muchas formas de resolver un factor significativo desde el punto de vista de la arquitectura, que se extienden desde omisiones benignas, a diseños extravagantes, o a productos de terceras partes.

En el UP, los factores de la arquitectura se recogen en la Especificación Complementaria, y las decisiones de diseño que los resuelven se recogen en el **Documento de la Arquitectura del Software** (SAD, *Software Architecture Document*, descrito con más detalle casi al final de este capítulo).

El análisis arquitectural comienza pronto durante la fase de inicio, y es uno de los puntos de interés de la fase de elaboración; es una actividad de alta prioridad y que tiene mucha influencia en el desarrollo de software. Este tema se ha aplazado hasta este

punto del libro de manera que se pudieran presentar en primer lugar los fundamentos del A/DOO. Es una actividad útil para:

- reducir el riesgo de olvidar algo esencial en el diseño de los sistemas
- evitar dedicar excesivo esfuerzo a cuestiones de poca prioridad
- ayudar a alinear el producto con los objetivos del negocio

Este capítulo es una introducción a los pasos e ideas básicas del análisis arquitectural desde la perspectiva del UP; es decir, al método, en lugar de a los consejos y astucias de los arquitectos expertos. De este modo, no es un libro de recetas de cocina de soluciones de arquitectura —un tema amplio y dependiente del contexto que va más allá del alcance de este libro introductorio—. No obstante, el caso de estudio del PDV NuevaEra que se comenta en el capítulo proporciona ejemplos concretos de soluciones relacionadas con la arquitectura.

32.1. Análisis arquitectural

El **análisis arquitectural** trata de la identificación y resolución de los requisitos no funcionales del sistema (por ejemplo, calidad), en el contexto de los requisitos funcionales.

En el UP, el término comprende tanto la investigación arquitectural (identificación) como el diseño arquitectural (resolución). A continuación se presentan algunos ejemplos de muchas de las cuestiones que se tienen que identificar y resolver al nivel de la arquitectura:

- ¿Cómo afectan en el diseño los requisitos de fiabilidad y tolerancia a fallos?
 - Por ejemplo, en el PDV NuevaEra, ¿para qué servicios remotos (ej. calculador de impuestos) se les podrá mantener el servicio ante fallos mediante servicios locales? ¿Por qué? ¿Proporcionan exactamente los mismos servicios localmente que de manera remota, o existen diferencias?
- ¿Cómo afecta en la rentabilidad el coste de las licencias de los subcomponentes comprados?
 - Por ejemplo, el fabricante del excelente servidor de bases de datos, *SinPistas*, quiere el 2% de cada PDV NuevaEra que se venda, si se utiliza su producto como subcomponente. La utilización de su producto acelerará el desarrollo (y su salida al mercado) porque es robusto y proporciona muchos servicios, y lo conocen muchos desarrolladores, pero esto tiene un precio. ¿En lugar de eso debería utilizar el equipo el servidor de bases de datos *SuSQL* de libre distribución, menos robusto? ¿Con qué riesgo? ¿Cómo limita la facultad de cobrar por el producto NuevaEra?
- ¿Cómo afecta la distribución de los servicios en los requisitos de calidad y los requisitos funcionales?
 - Por ejemplo, utilizando un sistema calculador de impuestos remoto (único y centralizado) reduce la huella de cada cliente NuevaEra, reduce los costes de li-

cencia (sólo se necesita una copia), y minimiza el esfuerzo de configuración para un cliente determinado (cada instalación requiere ajustes semanales debido a cambios en las políticas del negocio y en las de legislación). Sin embargo, el servicio remoto reduce el tiempo de respuesta lo suficiente para que los impuestos sólo se puedan calcular una vez, después de introducir todos los artículos; uno no puede ver un precio parcial con impuestos tras cada línea de venta; y la llamada remota tarda mucho. También crea un único punto de fallo.

- ¿Cómo afectan al diseño los requisitos de adaptabilidad y de configuración?
 - Por ejemplo, la mayoría de las tiendas difieren en las reglas del negocio que quieren representar en sus aplicaciones de PDV. ¿Cuáles son las variaciones? ¿Cuál es la “mejor” forma de diseñarlas? ¿Cuáles son los criterios para la mejor? ¿Puede NuevaEra ganar más dinero exigiendo que se adapte la programación a cada cliente (y, ¿cuánto esfuerzo supondrá?), o con una solución que permita a los clientes que las adapten ellos mismos fácilmente? ¿“Más dinero” debería ser el objetivo a corto plazo?

Pasos comunes en el análisis arquitectural

Existen varios métodos para el análisis arquitectural. Comunes a la mayoría de ellos son algunas variaciones de los siguientes pasos:

1. Identificar y analizar los requisitos no funcionales que influyen en la arquitectura. Los requisitos funcionales también son relevantes (especialmente en términos de variabilidad o cambio), pero se les presta una atención completa a los no funcionales. En general, todos éstos podrían llamarse **factores de la arquitectura** (también conocidos como **controladores de la arquitectura**, *architectural drivers*).
 - Este paso podría caracterizarse como un análisis de requisitos ordinario, pero puesto que se lleva a cabo en el contexto de la identificación del impacto de la arquitectura y en la toma de decisiones sobre las soluciones de la arquitectura de alto nivel, se considera como parte del análisis arquitectural en el UP.
 - Por lo que se refiere al UP, algunos de estos requisitos se identificarán y recopilarán en líneas generales en la Especificación Complementaria o en los casos de uso durante la fase de inicio. Durante el análisis arquitectural, que tiene lugar al principio de la elaboración, el equipo investiga estos requisitos más detenidamente.
2. Para aquellos requisitos que influyen de manera significativa en la arquitectura, analizar las alternativas y crear soluciones que resuelvan el impacto. Éstas son las **decisiones sobre la arquitectura**.
 - Las decisiones varían desde “eliminar el requisito”, a una solución a medida, a “detener el proyecto”, o a “contratar un experto”.

Esta presentación introduce estos pasos básicos en el contexto del caso de estudio del PDV NuevaEra. Por simplicidad, evita cuestiones del despliegue de la arquitectura tales

como la configuración del hardware y del sistema operativo, que son muy sensibles al contexto y al momento en que se hace.

32.2. Tipos y vistas de la arquitectura

Algunas de las descripciones de arquitectura definen tipos diferentes, como la “arquitectura de aplicación” (asignación de características a componentes) o “arquitectura del sistema” (configuración del hardware y del sistema operativo).

En el UP, existe una especialización de la información parecida, pero se describen en “vistas” de la arquitectura, que resumen y destacan una perspectiva particular. Por ejemplo, la **vista lógica** de la arquitectura, que se introdujo en el Capítulo 30, resume la organización y la funcionalidad de los elementos del software importantes (como las capas) —es similar al término arquitectura de la aplicación—. La **vista de despliegue** resume la topología del sistema, las comunicaciones y la correspondencia entre los elementos ejecutables y los nodos de proceso —es análogo al término arquitectura del sistema.

El UP define seis vistas de la arquitectura, que se describen en detalle casi al final de este capítulo. Concretamente, las vistas combinan texto y diagramas, y —si se describen del todo— se recogen en el SAD.

El análisis arquitectural se relaciona con las vistas de la arquitectura porque las decisiones sobre la arquitectura se reflejan y describen en una o más vistas de la arquitectura.

32.3. La ciencia: identificación y análisis de los factores de la arquitectura

Factores de la arquitectura

Cualquiera y todos los requisitos FURPS+ pueden influir de manera significativa en la arquitectura de un sistema, variando desde la fiabilidad, a la planificación, a las habilidades y a las restricciones de coste. Por ejemplo, un caso de planificación ajustada, habilidades limitadas y dinero suficiente probablemente favorece contratar o alquilar especialistas externos, en lugar de construir todos los componentes dentro de la compañía.

Sin embargo, los factores que influyen con más fuerza en la arquitectura tienden a encontrarse en las categorías FURPS+ de alto nivel de funcionalidad, fiabilidad, rendimiento, soporte, implementación e interfaz (véase el Capítulo 5 para un desglose detallado). Curiosamente, son los atributos de calidad no funcionales (como la fiabilidad o el rendimiento) los que dan a una arquitectura concreta su sabor único, en lugar de sus requisitos funcionales. Por ejemplo, el diseño en el sistema NuevaEra para dar soporte a diferentes componentes de terceras partes con interfaces únicas, y el diseño para dar soporte a la conexión fácil de diferentes conjuntos de reglas del negocio.

En el UP, estos factores con implicaciones en la arquitectura se denominan **requisitos significativos para la arquitectura**. Utilizamos aquí “factores” para simplificar.

Se pueden caracterizar muchos factores técnicos y organizativos como *restricciones* que limitan la solución de alguna manera (como, debe ejecutarse en Linux, o el presupuesto para comprar componentes de terceras partes es X).

Escenarios de calidad

Cuando se definen los requisitos de calidad durante el análisis de los factores de la arquitectura, se recomienda el uso de los **escenarios de calidad**¹, ya que definen respuestas cuantificables (o por lo menos observables) y, por tanto, se pueden verificar. No sirve de mucho establecer de manera vaga “el sistema será fácil de modificar” sin ninguna medida de lo que eso significa.

Cuantificar algunas cosas, como los objetivos de rendimiento y el tiempo entre fallos, son prácticas bien conocidas, pero los escenarios de calidad amplían esta idea y promueve la recopilación de todos (o al menos, la mayoría) los factores como sentencias cuantificables.

Los escenarios de calidad son sentencias cortas de la forma <estímulo> <respuesta cuantificable>; por ejemplo:

- Cuando se envía la venta completa al calculador de impuestos remoto para añadir los impuestos, el resultado se devolverá en 2 segundos “la mayoría” de las veces, medido en un entorno de producción bajo condiciones de carga “media”.
- Cuando llega un informe de errores de un voluntario de una prueba de una versión beta de NuevaEra, responda con una llamada de teléfono en un día laborable.

Nótese que será necesario que el arquitecto de NuevaEra realice un estudio adicional y defina “la mayoría” y “media”; un escenario de calidad no es realmente válido hasta que no se puede probar, lo que implica que se ha especificado completamente. También, observe la calificación en el primer escenario de calidad en función del entorno en el que se aplica. Esto mejora algo la especificación de un escenario de calidad, verifica que lo pasa en un entorno de desarrollo ligeramente cargado, pero falla al evaluarlo en un entorno de producción realista.

Escoja sus Batallas

Una advertencia: la escritura de estos escenarios de calidad puede ser un espejismo de utilidad. Es fácil *escribir* estas especificaciones detalladas, pero no materializarlas. ¿Alguien las probará realmente alguna vez? ¿Cómo y por quién? Se requiere una fuerte dosis de realismo cuando se escriben; no tiene sentido listar muchos objetivos sofisticados si realmente nunca nadie acabará comprobándolos.

Esto está relacionado con la discusión acerca de “escoja sus batallas” que se presentó en un capítulo anterior sobre el patrón Variaciones Protegidas. ¿Cuáles son realmente los escenarios de calidad críticos que causan el éxito o el fracaso? Por ejemplo, en un sistema de reservas de vuelos, es realmente crítico para el éxito del sistema que se completen las

¹ Un término utilizado en varios métodos de la arquitectura promovido por el Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*); por ejemplo, en el método de *Diseño Basado en la Arquitectura*.

transacciones de manera rápida y consistente bajo condiciones de carga alta —debe comprobarse sin lugar a dudas—. En el sistema NuevaEra, la aplicación realmente deber ser tolerante a fallos y mantener el servicio ante los fallos mediante copias locales de los servicios cuando fallan los remotos —sin duda se debe probar y validar debidamente—. Por tanto, céntrese en escribir los escenarios de calidad para las batallas importantes, y siga desde el principio hasta el final un plan para evaluarlos.

Descripción de los factores

Un objetivo importante del análisis arquitectural es entender la influencia de los factores, sus prioridades, y la manera en que varían (necesidad inmediata de flexibilidad y evolución futura). Por tanto, la mayoría de los métodos de la arquitectura (por ejemplo, véase [HNS00]) abogan por la creación de una tabla o árbol con variaciones de la siguiente información (el formato varía dependiendo del método). El siguiente estilo que se muestra en la Tabla 32.1 se denomina **tabla de factores**, que en el UP forma parte de la Especificación Complementaria.

Tabla 32.1. Ejemplo de tabla de factores.

Factor	Medidas y Escenarios de Calidad	Variabilidad (flexibilidad actual y futura evolución)	Impacto del factor (y su variabilidad) en las personas involucradas, arquitectura y otros factores	Prioridad para el éxito	Dificultad o Riesgo
Fiabilidad-Capacidad de recuperación					
Recuperación de los fallos en los servicios remotos	Cuando falla un servicio remoto, restablecer la conexión con él en 1 minuto, bajo una carga normal de la tienda en un entorno de producción.	Flexibilidad actual: nuestro EME dice que son aceptables (y convenientes) los servicios simplificados locales del lado del cliente hasta que sea posible la re-conexión. Evolución: en 2 años, algunas tiendas podrían tener la intención de pagar por una copia local completa de los servicios remotos (como el calculador de impuestos). ¿Probabilidad? Alta.	Impacto alto en el diseño a gran escala. A las tiendas realmente les disgusta cuando fallan los servicios remotos, ya que les impide o restringe el uso de un PDV para realizar las ventas.	A	M
...	...	...	...		

Leyenda: A: Alta. M: Media. EME: Experto en la Materia de Estudio.

Obsérvese el esquema de clasificación: *Fiabilidad—Capacidad de recuperación* (a partir de las categorías del FURPS+). Éste no se presenta como el mejor o el único esquema, pero resulta útil para agrupar los factores de la arquitectura en categorías. Por ejemplo,

ciertas categorías (como fiabilidad y rendimiento) están fuertemente relacionadas con la identificación y definición de los planes de pruebas y, por tanto, es conveniente agruparlas.

Los valores de los códigos básicos para la prioridad y el riesgo de A/M/B simplemente insinúan que se utilicen algunos códigos que el equipo encuentre útiles; existe una variedad de esquemas de codificación (numéricos y cualitativos) procedentes de diferentes métodos y estándares (como el ISO 9126) de arquitectura. Una advertencia: Si el esfuerzo extra de utilizar un esquema más complejo no le conduce a ninguna acción práctica, no merece la pena.

Factores y los artefactos del UP

El repositorio central de requisitos funcionales en el UP son los casos de uso, y ellos, junto con la Visión y la Especificación Complementaria, son una fuente de inspiración importante cuando se está creando una tabla de factores. En los casos de uso, se deberían revisar los *Requisitos especiales*, las *Variaciones de la tecnología* y los *Temas abiertos*, y reunir sus factores de la arquitectura implícitos o explícitos en la Especificación Complementaria.

Es razonable recoger al principio los factores relacionados con casos de uso en los casos de uso mientras se están creando, debido a que existe una relación obvia, pero al final es más conveniente (en cuanto a la gestión del contenido, traza y legibilidad) reunir todos los factores de la arquitectura en un sitio —en la tabla de factores en la Especificación Complementaria—.

Caso de Uso UC1: Procesar Venta

Escenario principal de éxito:

1. ...

Requisitos especiales:

- El tiempo de respuesta para la autorización de crédito será de 30 segundos el 90% de las veces
- De algún modo, queremos que el sistema se recupere de manera robusta cuando falla el acceso a servicios remotos, como el sistema de inventario.
- ...

Lista de tecnología y variaciones de datos:

2a. El identificador del artículo se introduce por medio de un escáner láser de código de barras (si está presente el código de barras) o del teclado.

...

Temas abiertos:

- ¿Cuáles son las variaciones en la ley de impuestos?
- Estudiar las cuestiones sobre la recuperación de los servicios remotos.

32.4. Ejemplo: tabla de factores parcial de la arquitectura del PDV NuevaEra

La tabla de factores parcial de la Tabla 32.2 muestra algunos factores relacionados con la discusión posterior.

Tabla 32.2. Tabla de factores parcial para el análisis arquitectural de NuevaEra.

Factor	Medidas y Escenarios de Calidad	Variabilidad (flexibilidad actual y futura evolución)	Impacto del factor (y su variabilidad) en las personas involucradas, arquitectura y otros factores	Prioridad para el éxito	Dificultad o Riesgo
Fiabilidad—Capacidad de recuperación					
Recuperación de los fallos en los servicios remotos.	Cuando falla un servicio remoto, restablecer la conexión con él en 1 minuto, bajo una carga normal de la tienda en un entorno de producción.	Flexibilidad actual: nuestro EME dice que son aceptables (y convenientes) los servicios simplificados locales del lado del cliente hasta que sea posible la reconexión. Evolución: en 2 años, algunas tiendas podrían tener la intención de pagar por una copia local completa de los servicios remotos (como el calculador de impuestos). ¿Probabilidad? Alta.	Impacto alto en el diseño a gran escala. A las tiendas realmente les disgusta cuando fallan los servicios remotos, ya que les impide o restringe el uso de un PDV para realizar las ventas.	A	M
Recuperación de los fallos en bases de datos de productos remotas.	Como arriba.	Flexibilidad actual: nuestro EME dice que es aceptable (y conveniente) que se almacene en el lado del cliente información acerca de los productos “más comunes” hasta que sea posible la reconexión. Evolución: en 3 años, las soluciones de almacenamiento masivo y replicación en el lado del cliente, serán baratas y efectivas, permitiendo que se mantenga una copia completa permanente y así el uso local. ¿Probabilidad? Alta.	Como arriba.	A	M
Soporte—Adaptabilidad					
Dar soporte a muchos servicios de terceras partes (calculador de impuestos, inventario, RRHH, contabilidad). Variarán en cada instalación.	Cuando se deba integrar un nuevo servicio de terceras partes, se puede hacer con un esfuerzo de 10 días persona.	Flexibilidad actual: como se describe en el factor. Evolución: ninguna.	Se requiere para la aprobación del producto. Poco impacto en el diseño.	A	B

(Continúa)

Tabla 32.2. Tabla de factores parcial para el análisis arquitectural de NuevaEra. (Continuación)

Factor	Medidas y Escenarios de Calidad	Variabilidad (flexibilidad actual y futura evolución)	Impacto del factor (y su variabilidad) en las personas involucradas, arquitectura y otros factores	Prioridad para el éxito	Dificultad o Riesgo
Soporte—Adaptabilidad (continuación)					
¿Dar soporte a terminales PDA inalámbricos para los clientes del PDV?	Cuando se añade el soporte a estos terminales, no requiere que se cambie el diseño de las capas de la arquitectura que no tienen que ver con el UI.	Flexibilidad actual: no se requiere en este momento. Evolución: en 3 años, pensamos que es muy probable que el mercado demande “PDAs” inalámbricos para los clientes del PDV.	Impacto alto en el diseño en cuanto a las variaciones protegidas de muchos elementos. Por ejemplo, los sistemas operativos y las UIs son diferentes en dispositivos pequeños.	B	A
Otros—Legal					
Se deben aplicar la legislación de impuestos actual.	Cuando el auditor evalúe si se ajusta, encontrará que se ajusta al 100%. Cuando cambia la legislación de los impuestos, estarán operativas dentro del plazo marcado por el gobierno.	Flexibilidad actual: la conformidad es inflexible, pero la legislación de los impuestos pueden cambiar casi semanalmente debido a que hay muchas leyes y niveles de impuestos del gobierno (nacional, regional,...). Evolución: ninguna.	Fallar en el cumplimiento es un delito. Influye en los servicios de cálculo de impuestos. Es difícil escribir nuestro propio servicio —leyes complejas, cambios constantes, la necesidad de seguir la pista a todos los niveles de gobierno—. Aunque, riesgo fácil/bajo si se compra un paquete.	A	B

Leyenda: A: Alta; M: Media; B: Baja; EME: Experto en la Materia de Estudio.

32.5. El arte: resolución de los factores de la arquitectura

Uno podría decir que la *ciencia* de la arquitectura es la recolección y organización de la información sobre los factores de la arquitectura, como en la tabla de factores. El *arte* de la arquitectura es tomar las decisiones acertadas para resolver estos factores, teniendo en cuenta los compromisos, interdependencias y prioridades.

Los arquitectos expertos conocen una variedad de áreas (por ejemplo, estilos y patrones de arquitectura, tecnologías, productos, peligros y tendencias) y las aplican a sus decisiones.

Registro de alternativas, decisiones y la motivación de la arquitectura

Ignorando por ahora los principios de la toma de decisiones sobre la arquitectura, casi todos los métodos de arquitectura recomiendan mantener un registro de las soluciones alternativas, decisiones, factores que influyen, y el motivo de las cuestiones y decisiones relevantes.

A tales registros se les ha llamado **memorándums técnicos** [Cunningham96], **tarjetas de cuestiones** [HNS00], y **documentos de técnicas de arquitectura** (propuestas de arquitectura del SEI), con diversos grados de formalidad y sofisticación. En algunos métodos, estos memorándums son la base para todavía otro paso de revisión y refinamiento.

En el UP, los memorándums se deben recoger en el SAD.

Un aspecto importante de los memorándums técnicos es la *motivación* o los fundamentos. Cuando un futuro desarrollador o arquitecto necesita modificar el sistema², es enormemente útil entender los motivos que están detrás del diseño, como *por qué* se escogió un enfoque concreto para la recuperación de los fallos de los servicios remotos en el PDV NuevaEra y se rechazaron otros, con el objeto de tomar decisiones con conocimiento de causa sobre los cambios del sistema.

Es importante explicar los motivos para rechazar las alternativas, ya que durante la evolución futura del producto, un arquitecto podría reconsiderar estas alternativas, o por lo menos querer conocer qué alternativas se consideraron, y por qué se escogió una.

A continuación presentamos un memorándum técnico de ejemplo que recoge una decisión sobre la arquitectura para el PDV NuevaEra. Por supuesto, el formato exacto no es importante. Manténgalo simple y recoja únicamente la información que ayudará a los futuros lectores a tomar decisiones con conocimiento de causa cuando estén cambiando el sistema.

Memorándum Técnico

Asunto: Fiabilidad—Recuperación de fallos en los servicios remotos

Resumen de la solución: Ubicación transparente utilizando un servicio de búsqueda, mantenimiento del servicio ante los fallos pasando de remoto a local, y replicación parcial local del servicio.

Factores

- Recuperación robusta de los fallos en los servicios remotos (ej. calculador de impuestos, inventario)

² ¡O cuando han pasado cuatro semanas y el arquitecto original ha olvidado su propia motivación!

- Recuperación robusta de los fallos en la base de datos de productos (ej. descripción de productos y precios) remota

Solución

Conseguir variaciones protegidas con respecto a la ubicación de los servicios utilizando un Adaptador creado en una FactoriaDeServicios. Donde sea posible, ofrecer implementaciones locales de servicios remotos, normalmente con comportamiento simplificado o restringido. Por ejemplo, el calculador de impuestos local utilizará porcentajes de impuestos constantes. La base de datos con información de los productos local será una pequeña caché de los productos más comunes. Se almacenarán las actualizaciones del inventario y se remitirán cuando se restablezca la conexión.

Véase también el memorándum técnico de *Adaptabilidad—Servicios de terceras partes* para conocer los aspectos de adaptabilidad de esta solución, porque las implementaciones de los servicios remotos variarán en cada instalación.

Para satisfacer los escenarios de calidad de reconexión con los servicios remotos ASAP, utilizar para los servicios objetos Proxy inteligentes, que en cada llamada a un servicio comprueban si se puede reactivar el servicio remoto y lo redireccionan a éste cuando es posible.

Motivación

¡Las tiendas realmente no quieren dejar de vender! Por tanto, si el PDV NuevaEra ofrece este nivel de fiabilidad y recuperación, será un producto muy atractivo, ya que ninguno de nuestros competidores proporciona esta capacidad. La pequeña caché de productos está motivada por la gran limitación de recursos en el lado del cliente. El calculador de impuestos de terceras partes real no se duplica en el cliente fundamentalmente debido a que el coste de las licencias es más alto y a los esfuerzos de configuración (puesto que cada instalación del calculador requiere ajustes casi semanales). Este diseño también soporta los puntos de evolución de los deseos de futuros clientes y es capaz de replicar los servicios permanentemente como el calculador de impuestos en cada terminal del cliente.

Cuestiones sin resolver

Ninguna

Alternativas consideradas

Una calidad de “nivel oro” del contrato de servicio con los servicios de autorización de crédito para mejorar la fiabilidad. Estaba disponible, pero era demasiado caro.

Nótese como se ilustra en este ejemplo —y es un punto clave— que una decisión sobre la arquitectura descrita en un memorándum técnico podría resolver un grupo de factores, no sólo uno.

Prioridades

Hay una jerarquía de objetivos que guían las decisiones sobre la arquitectura:

1. Restricciones inflexibles, entre las que se encuentran ajustarse a las normas legales y de seguridad.
 - El PDV NuevaEra debe aplicar correctamente las leyes de impuestos.
2. Objetivos del negocio.
 - Demo de las características relevantes listo para la feria POSWorld de Hamburgo dentro de 18 meses.

- Tiene cualidades y características atractivas para los grandes almacenes de Europa (por ejemplo, que soporte diferentes monedas y reglas de negocio a medida).
3. Todos los otros objetivos.
- A menudo se les pueden seguir la pista hacia atrás hasta establecer directamente objetivos del negocio, pero son indirectos. Por ejemplo, se podría establecer la traza entre “fácilmente extensible: puede añadir <alguna unidad funcional> en 10 semanas persona” y el objetivo del negocio de “nueva versión cada seis meses”.

En el UP, muchos de estos objetivos se recogen en el artefacto de Visión. Recuerde que los valores de la *Prioridad para el éxito* en la tabla de factores debería reflejar la prioridad de estos objetivos.

Existe un aspecto distintivo de la toma de decisiones a este nivel frente al diseño de objetos a pequeña escala: uno tiene que considerar simultáneamente más objetivos (y a menudo que influyen globalmente) y sus compromisos. Además, los objetivos del negocio pasan a ser cruciales para las decisiones técnicas (o al menos deberían). Por ejemplo:

Memorándum Técnico

Asunto: Legal—Cumplimiento de la legislación de impuestos

Resumen de la solución: Comprar un componente para el cálculo de impuestos.

Factores

- Se deben aplicar, por ley, la legislación de impuestos actuales.

Solución

Comprar un calculador de impuestos con un acuerdo de licencia para recibir actualizaciones de la legislación de impuestos actuales. Nótese que se podrían utilizar diferentes calculadores en instalaciones distintas.

Motivación

Acelerar la salida al mercado, corrección, requisitos de mantenimiento bajos, y desarrolladores felices (véase las alternativas). Estos productos son costosos, lo que afecta a nuestros objetivos del negocio de política de contención de costes y de política de fijación de precios, pero la alternativa se considera inaceptable.

Cuestiones sin resolver

¿Cuáles son los productos líderes y sus cualidades?

Alternativas consideradas

¿Qué el equipo de NuevaEra construya uno? Se estima que puede llevar mucho tiempo, ser propenso a fallos, y crea una responsabilidad de mantenimiento continua, costosa y sin interés (para los desarrolladores de la compañía), que afecta al objetivo de “desarrolladores felices” (seguramente, el objetivo más importante de todos).

Prioridades y puntos de evolución: ingeniería por defecto o en exceso

Otra característica distintiva de la toma de decisiones sobre la arquitectura es establecer prioridades de acuerdo a la probabilidad de los **puntos de evolución** —puntos de variabilidad o cambio que *podrían* surgir en el futuro—. Por ejemplo, en NuevaEra, existe una posibilidad de que se desee terminales clientes portátiles e inalámbricos. El diseño para esto influye de manera significativa debido a las diferencias en los sistemas operativos, interfaces de usuario, recursos hardware, etcétera.

La compañía podría gastarse una cantidad enorme de dinero (e incrementar diversos riesgos) para conseguir esta “futura necesidad”. Si en el futuro resulta que no es relevante, hacerlo sería un ejercicio muy caro de sobre-ingeniería. Nótese también que se puede sostener que las futuras necesidades rara vez se dan, puesto que son especulaciones; incluso si ocurre el cambio previsto, es probable que se produzcan algunos cambios en el diseño supuesto.

Por otro lado, las futuras necesidades contra el problema de datos Y2K habría sido dinero bien empleado; en lugar de eso, hubo un esfuerzo de ingeniería por defecto con un resultado tremendamente costoso.

El arte del arquitecto es conocer qué batallas merece la pena pelear —dónde merece la pena invertir en diseños que protejan contra cambios evolutivos—.

Para decidir si se deberían evitar “futuras necesidades” prematuras, considere realmente el escenario de posponer el cambio para el futuro, cuando se requiera. ¿Cuánto del diseño y del código tendrá que cambiar en realidad? ¿Cuál será el esfuerzo? Quizás un estudio detallado del cambio potencial revelará que lo que al principio se consideró una cuestión gigantesca contra la que protegerse, se estima que el esfuerzo será sólo de unas pocas semanas-persona.

Esto es precisamente un problema difícil: “La predicción es muy difícil, especialmente si trata sobre el futuro” (atribuido, aunque sin comprobar, a Niels Bohr).

Principios básicos de diseño de la arquitectura

Los principios básicos de diseño que se han estudiado en gran parte de este libro que se aplicaron al diseño de objetos a pequeña escala, todavía son principios de gran influencia al nivel de la arquitectura a gran escala:

- Bajo acoplamiento.
- Alta cohesión.
- Variaciones protegidas (interfaces, indirección, servicio de búsqueda, etcétera).

Sin embargo, la granularidad de los componentes es mayor —se trata de bajo acoplamiento entre aplicaciones, subsistemas, o procesos, en lugar de entre pequeños objetos—.

Además, a esta escala más amplia, hay más o diferentes mecanismos para conseguir cualidades como el bajo acoplamiento y variaciones protegidas. Por ejemplo, considere el siguiente memorándum técnico:

Memorándum Técnico

Asunto: Adaptabilidad—Servicios de terceras partes

Resumen de la solución: Variaciones Protegidas utilizando interfaces y adaptadores.

Factores

- Dar soporte a muchos y cambiables servicios de terceras partes (calculadores de impuestos, autorización de crédito, inventario...).

Solución

Conseguir variaciones protegidas como sigue: Analizar varios productos para el cálculo de impuestos comerciales (y así sucesivamente para las otras categorías de productos) y construir interfaces comunes para el mínimo común denominador de funcionalidades. Entonces utilizar la Indirección mediante el patrón Adaptador. Es decir, crear un objeto Adaptador de recursos que implementa la interfaz y actúa como conexión y traductor con un calculador de impuestos back-end concreto.

Véase también el memorándum técnico *Fiabilidad—Recuperación de fallos en los servicios remotos* para conocer los aspectos de la ubicación transparente de esta solución.

Motivación

Simple. Comunicación más barata y más rápida que utilizando un servicio de intercambio de mensajes (ver alternativas), y en cualquier evento, un servicio de intercambio de mensajes no se puede utilizar para conectar directamente con el servicio de autorización de crédito externo.

Cuestiones sin resolver

¿Originarán las interfaces del mínimo común denominador un problema imprevisto, tal como ser demasiado limitada?

Alternativas consideradas

Aplicar indirección utilizando un servicio de intercambio de mensajes o publicar-suscribir (ej. una implementación JMS) entre el cliente y el calculador de impuestos, con adaptadores. Pero no se puede utilizar directamente con un autorizador de crédito, costoso (para los fiables), y más fiabilidad en la entrega de mensajes de lo que se necesita en la práctica.

El hecho es que al nivel de la arquitectura, existen normalmente nuevos mecanismos para conseguir variaciones protegidas (y otros objetivos), con frecuencia en colaboración con componentes de terceras partes, como utilizar el Servicio de Mensajes de Java (JMS; *Java Messaging Service*) o el servidor EJB.

Separación de intereses y localización del impacto

Otro principio básico que se aplica durante el análisis arquitectural es conseguir **separación de intereses**. Esto también es aplicable en la escala de pequeños objetos, pero alcanza relevancia durante el análisis arquitectural.

Los **intereses transversales** son aquéllos con amplia aplicación o influencia en el sistema, como la persistencia de datos o la seguridad. Uno *podría* diseñar el soporte a la persistencia en la aplicación NuevaEra de manera que cada objeto (que contiene código de la lógica de la aplicación) se comunique él mismo con una base de datos para almacenar sus datos. Estó entrelazaría el interés de la persistencia con el de la lógica de la

aplicación, en el código fuente de las clases —así también con la seguridad—. La cohesión disminuye y aumenta el acoplamiento.

En cambio, diseñando para conseguir una separación de intereses se separa el soporte a la persistencia y el soporte a la seguridad en “cosas” separadas (existen mecanismos muy diferentes para esta separación). Un objeto con lógica de la aplicación únicamente tiene lógica de la aplicación, no lógica de persistencia o de seguridad. Igualmente, un subsistema de persistencia se centra en las cuestiones de persistencia, no de seguridad. Un subsistema de seguridad no lleva a cabo la persistencia.

La separación de intereses es una forma de pensar a gran escala acerca del bajo acoplamiento y alta cohesión al nivel de arquitectura. También se aplica a objetos a pequeña escala, porque su ausencia da lugar a objetos sin cohesión que tienen muchas áreas de responsabilidad. Pero es una cuestión de arquitectura especialmente porque los intereses son amplios, y las soluciones conllevan elecciones de diseño importantes y fundamentales.

Existen por lo menos tres técnicas a gran escala para conseguir la separación de intereses:

1. Tratar el interés como un módulo en un componente separado (por ejemplo, un subsistema) e invocar sus servicios.
 - Éste es el enfoque más común. Por ejemplo, en el sistema NuevaEra, el soporte a la persistencia podría colocarse en un subsistema aparte denominado *servicio de persistencia*. Mediante una fachada, puede ofrecer una interfaz público de servicios a otros componentes. Las arquitecturas en capas también ilustran esta separación de intereses.
2. Utilizar decoradores.
 - Éste es el segundo enfoque más común; se dio a conocer en primer lugar en el Servicio de Transacciones de Microsoft, y posteriormente con los servidores EJB. En este enfoque, un interés (como la seguridad) decora otros objetos con un objeto Decorador que encapsula el objeto interno e interpone el servicio. Al Decorador se le denomina **contenedor** en la terminología EJB. Por ejemplo, en el sistema de PDV NuevaEra, el control de seguridad a los servicios externos como el sistema de RRHH se puede conseguir con un contenedor EJB que añade las comprobaciones de seguridad en el Decorador externo, alrededor de la lógica de la aplicación del objeto interno.
3. Utilizar post-compiladores y tecnologías orientadas a aspectos.
 - Por ejemplo, con entidades *bean* EJB uno puede añadir el soporte a la persistencia a clases como *Venta*. Uno especifica en un fichero de descripción de propiedades las características de persistencia de la clase *Venta*. Entonces, un post-compilador (que es otro compilador que se ejecuta después de un compilador “ordinario”) añadirá el soporte a la persistencia necesario en una clase *Venta* modificada (modificando únicamente los byte-codes) o una subclase. El desarrollador continúa viendo la clase original como una clase “limpia” que sólo contiene la lógica de la aplicación. Otra variación son las tecnologías **orientadas a aspectos** como AspectJ

(www.aspectj.org), que de manera similar da soporte a entrelazar post-compilación de intereses transversales en el código, de manera transparente para el desarrollador. Estos enfoques mantienen la ilusión de la separación durante el trabajo de desarrollo, y entrelazan el interés antes de la ejecución.

Promoción de patrones de arquitectura

Un estudio de los patrones de arquitectura y del modo en el que se podrían aplicar (o aplicar mal) al caso de estudio NuevaEra queda fuera del alcance de este texto introductorio. Sin embargo, a continuación presentamos algunas indicaciones:

Probablemente el mecanismo más común para conseguir bajo acoplamiento, variaciones protegidas y separación de intereses al nivel de la arquitectura es el patrón Capas, que se introdujo en un capítulo anterior. Éste es un ejemplo de la técnica más común de separación —tratar los intereses como módulos, en componentes separados o capas—.

Existe un amplio y creciente cuerpo de material escrito sobre patrones de arquitectura. Estudiarlos es la manera más rápida que conozco de aprender soluciones de la arquitectura. Por favor, vea las lecturas recomendadas.

32.6. Resumen de los temas del análisis arquitectural

Un asunto a destacar es que las cuestiones “arquitecturales” están especialmente relacionadas con los requisitos no funcionales, y conllevan la percepción del contexto del negocio o del mercado de la aplicación. Al mismo tiempo, no se pueden ignorar los requisitos funcionales (por ejemplo, procesar ventas); éstos proporcionan el contexto en el que se deben resolver estas cuestiones. Por otro lado, la identificación de su variabilidad es significativa para la arquitectura.

Un segundo tema es que las cuestiones arquitecturales implican problemas al nivel del sistema, a gran escala y amplios, cuya resolución normalmente conlleva decisiones de diseño a gran escala o fundamentales; por ejemplo, la elección de —o incluso el uso de— un servidor de aplicación.

Un tercer tema en el análisis arquitectural son las interdependencias y compromisos. Por ejemplo, la mejora de la seguridad podría afectar al rendimiento o a la facilidad de uso, y la mayoría de las opciones afectan al coste.

Un cuarto tema del análisis arquitectural es la generación y evaluación de soluciones alternativas. Un arquitecto experto puede ofrecer soluciones de diseño que impliquen la construcción de nuevo software, y también sugerir soluciones (o soluciones parciales) que utilicen software y hardware comercial o de libre distribución. Por ejemplo, la recuperación en un servidor remoto del PDV NuevaEra se puede conseguir mediante el diseño y programación de procesos “guardianes (*watchdog*)”, o quizás a través de la agrupación, duplicación y los servicios para sobreponerse a fallos que ofrecen algunos sistemas operativos y componentes hardware. Los buenos arquitectos conocen los productos hardware y software de terceras partes.

La definición inicial de cuestiones de la arquitectura proporciona el marco para el modo de pensar sobre el tema de la arquitectura: identificando los puntos con implicaciones a gran escala o a nivel del sistema, y resolviéndolos.

Un análisis arquitectural se preocupa de la identificación y resolución de los requisitos no funcionales (p.e. calidad) del sistema, en el contexto de los requisitos funcionales.

32.7. Análisis arquitectural en el UP

Advertencia: Análisis arquitectural en cascada

Con frecuencia, los métodos y libros sobre el análisis arquitectural implícitamente fomentan extensas decisiones de diseño sobre la arquitectura siguiendo un estilo en cascada antes de la implementación. En el desarrollo iterativo y el UP, aplique estas ideas en el contexto de pequeñas etapas, retroalimentación y adaptación, en lugar de pretender resolver completamente la arquitectura antes de programar. Aborde la implementación de las soluciones más arriesgadas o más difíciles en las primeras iteraciones, y ajuste las soluciones sobre la arquitectura en base a la retroalimentación y al conocimiento que adquiere.

Información sobre la arquitectura en los artefactos del UP

- Los factores de la arquitectura (por ejemplo, en una tabla de factores) se recogen en la Especificación Complementaria.
- Las decisiones sobre la arquitectura se recogen en el SAD. Esto incluye los memorándums técnicos y las descripciones de las vistas de la arquitectura.

El SAD y sus vistas de la arquitectura

Además de los diagramas UML de paquetes, clases e interacciones, el SAD es otro artefacto clave del Modelo de Diseño del UP. Éste describe las grandes ideas de la arquitectura, incluyendo las decisiones sobre el análisis arquitectural. Prácticamente, es una *ayuda de aprendizaje* para los desarrolladores que necesitan entender las ideas esenciales del sistema.

Cuando alguien se une al equipo, un jefe de proyecto puede decir, “¡Bienvenido al proyecto NuevaEra! Por favor, vaya al sitio web del proyecto y lea el SAD de 10 páginas para obtener una visión de las ideas más importantes”. Durante una versión posterior, cuando trabaja en el sistema gente nueva, el SAD constituye una ayuda clave para el aprendizaje.

Por tanto, se debe escribir teniendo en mente quien lo leerá y el objetivo: ¿qué necesito decir (y representar en UML) que rápidamente ayudará a alguien a entender las ideas principales del sistema?

La esencia del SAD es un resumen de las decisiones sobre la arquitectura (como con los memorándums técnicos) y las vistas de la arquitectura del UP.

Vistas de la arquitectura en el SAD

Tener una arquitectura es una cosa, describirla es algo más.

En [Kruchten95], se fomenta la influyente idea de describir una arquitectura con múltiples vistas. La idea esencial de una **vista de la arquitectura** es ésta:

Vista de la arquitectura

Una vista de la arquitectura del sistema desde una perspectiva dada; se centra sobre todo en la estructura y modularidad de los componentes fundamentales y en los principales flujos de control [RUP].

Un aspecto importante de la vista que se obvia en esta definición del RUP es la *motivación*. Es decir, una vista de la arquitectura debería explicar por qué la arquitectura es como es.

Una vista de la arquitectura es una ventana sobre el sistema desde una perspectiva particular que destaca la información relevante o ideas claves, e ignora el resto.

Una vista de la arquitectura es una herramienta de comunicación, enseñanza o de reflexión; se representa con texto y diagramas UML.

En el UP, se sugieren seis vistas de la arquitectura (se permiten más, como una vista de seguridad)³. Todas son opcionales, pero se recomienda documentar por lo menos las vistas lógica, de proceso, de caso de uso, y de despliegue. Las seis vistas son:

1. Lógica

- Organización conceptual del software en función de las capas, subsistemas, paquetes, frameworks, clases e interfaces más importantes. También resume la funcionalidad de los elementos del software importantes, como cada subsistema.
- Muestra los escenarios de realización de casos de uso (como diagramas de interacción) destacados que ilustran los aspectos claves del sistema.
- Una vista sobre el Modelo de Diseño del UP, visualizada con diagramas de paquetes, clases e interacción de UML.

2. Proceso

- Procesos e hilos de ejecución. Sus responsabilidades, colaboraciones y la asignación a ellos de los elementos lógicos (capas, subsistemas, clases,...).
- Una vista sobre el Modelo de Diseño del UP, visualizada con diagramas de clases e interacción de UML, utilizando la notación UML para procesos e hilos.

³ Las primeras versiones del UP describían las “4+1” vistas como se definen en [Kruchten95], que evolucionaron a las seis vistas.

3. Despliegue

- Despliegue físico de los procesos y componentes sobre los nodos de proceso, y la configuración de la red física entre los nodos.
- Una vista sobre el Modelo de Despliegue del UP, visualizada con los diagramas de despliegue de UML. Normalmente, la “vista” es simplemente el modelo completo en lugar de un subconjunto, ya que todo es relevante. Véase el Capítulo 38 para conocer la notación para el diagrama de despliegue de UML.

4. Datos

- Vista global del esquema de datos persistentes, la correspondencia del esquema de objetos a datos persistentes (normalmente en una base de datos relacional), el mecanismo de correspondencia de objetos a una base de datos, procedimientos almacenados en la base de datos y disparadores (*triggers*).
- Una vista sobre el Modelo de Datos del UP, visualizada con diagramas de clases de UML que se utilizan para describir un modelo de datos.

5. Casos de uso

- Resumen de los casos de uso más significativos para la arquitectura y sus requisitos no funcionales. Es decir, aquellos casos de uso que, mediante su implementación, cubren una parte significativa de la arquitectura o que influyen en muchos elementos de la arquitectura. Por ejemplo, el caso de uso *Procesar Venta*, cuando se implementa completamente, tiene estas cualidades.
- Una vista sobre el Modelo de Casos de Uso del UP, expresada textualmente y visualizada con los diagramas de casos de uso de UML.

6. Implementación

- En primer lugar, una definición del Modelo de Implementación: a diferencia de otros modelos del UP, que son texto y diagramas, este “modelo” es el código fuente real, ejecutables, etcétera. Tiene dos partes: 1) entregables, y 2) cosas que crean a los entregables (como código fuente y gráficos). El Modelo de Implementación está formado por todas estas cosas, incluyendo las páginas web, DLLs, ejecutables, código fuente, etcétera, y su organización —como el código fuente en los paquetes de Java y bytecodes organizados en ficheros JAR—.
- La vista de implementación es una descripción resumida de la organización relevante de los entregables y de las cosas que crean a los entregables (como el código fuente).
- Una vista sobre el Modelo de Implementación del UP, expresada textualmente y visualizada con los diagramas de paquetes y componentes de UML.

Por ejemplo, los diagramas de paquetes y de interacción de NuevaEra que se presentaron en el Capítulo 30 sobre la arquitectura en capas y lógica, muestran las grandes ideas de la estructura lógica de la arquitectura del software. En el SAD, el arquitecto creará una sección denominada *Vista Lógica*, insertará estos diagramas UML, y añadirá

algunos comentarios escritos acerca de para qué es cada paquete y capa, y el motivo que hay detrás del diseño lógico. De igual modo con las vistas de proceso y despliegue.

Una idea clave de las vistas de la arquitectura —que concretamente son texto y diagramas— es que *no* describen *todo* el sistema desde alguna perspectiva, sino sólo ideas destacadas desde esa perspectiva. Una vista es, si se quiere, la descripción “en un minuto de ascensor”: ¿Cuáles son las cosas más importantes que dirías en un minuto en un ascensor a un compañero sobre esta perspectiva?

Podrían crearse vistas de la arquitectura:

- después de que se construya el sistema, como resumen o ayuda de aprendizaje para futuros desarrolladores
- al final de ciertos hitos de la iteración (como al final de la iteración) para que sirva de ayuda para el aprendizaje para el equipo de desarrollo actual, y nuevos miembros
- de manera especulativa, durante las primeras iteraciones, como ayuda en el trabajo de diseño creativo, reconociendo que la vista original cambiará cuando prosiga el diseño y la implementación.

Estructura de ejemplo de un SAD

Documento de la Arquitectura del Software

Representación de la arquitectura

(Resumen del modo en el que se describirá la arquitectura en este documento, como utilizando memorándums técnicos y las vistas de la arquitectura. Esto es útil para alguien que no esté familiarizado con la idea de los memorándums técnicos o las vistas. Nótese que no son necesarias todas las vistas.)

Factores y decisiones de la arquitectura

(Referencia a la Especificación Complementaria para ver la Tabla de Factores. También, el conjunto de memorándums técnicos que resume las decisiones.)

Vista Lógica

(Los diagramas de paquetes de UML y los diagramas de clases de los elementos importantes. Comentarios sobre la estructura a gran escala y la funcionalidad de los componentes principales.)

Vista Proceso

(Diagramas de interacción y de clases de UML que ilustran los procesos e hilos de ejecución del sistema. Agruparlos de acuerdo a los hilos y procesos que interaccionan. Comentarios sobre el modo en que funciona la comunicación entre los procesos (ej. mediante RMI de Java).

Vista de Casos de Uso

(Resumen breve de los casos de uso más significativos desde el punto de vista de la arquitectura. Diagramas de interacción UML para algunas de las realizaciones de los casos de uso significativos para la arquitectura, o escenarios, con comentarios en los diagramas explicando cómo ilustran los elementos importantes de la arquitectura.)

Vista de Despliegue

(Los diagramas de despliegue de UML que muestra los nodos y la asignación de los procesos y componentes. Comentarios sobre la red.)

Fases

Inicio: Si no está claro si es técnicamente posible satisfacer los requisitos significativos de la arquitectura, el equipo debe implementar una **prueba de concepto de la arquitectura** (PDC) para determinar la viabilidad. En el UP, su creación y evaluación se denomina **Síntesis de la Arquitectura**. Esto es distinto de los anteriores experimentos pequeños y simples de programación de PDC para cuestiones técnicas aisladas. Una PDC de la arquitectura cubre ligeramente *muchos* de los requisitos significativos para la arquitectura para evaluar su viabilidad *combinada*.

Elaboración: Un objetivo importante de esta fase es la implementación de los elementos centrales de la arquitectura de riesgo, de esta manera la mayoría del análisis arquitectural se completa durante la elaboración. Normalmente se espera que la mayor parte del contenido de la tabla de factores, memorándums técnicos y el SAD se pueda completar al final de la elaboración.

Transición: Aunque idealmente los factores y decisiones significativas para la arquitectura se resolvieron mucho antes de la transición, el SAD necesitará que se repase y posiblemente que se revise al final de esta fase para asegurar que describe de manera precisa el sistema de despliegue final.

Ciclos de evolución siguientes: Antes de diseñar nuevas versiones, es común volver a visitar los factores de la arquitectura y las decisiones. Por ejemplo, la decisión de la versión 1.0 de crear un único servicio remoto para el cálculo de impuestos, en lugar de un duplicado en cada nodo de PDV, podría haberse motivado por el coste (para evitar múltiples licencias). Pero quizás en el futuro, se reduce el coste de los calculadores de impuestos, y de esta manera, por razones de tolerancia a fallos o de rendimiento, la arquitectura cambia para utilizar varios calculadores de impuestos.

32.8. Lecturas adicionales

Existe un cuerpo creciente de patrones relacionados con la arquitectura, y consejos generales sobre la arquitectura del software. Sugerencias:

- *Pattern-Oriented Software Architecture*, ambos volúmenes.
- *Software Architecture in Practice* [BCK98].
- *Pattern Languages of Program Design*, todos los volúmenes. Cada volumen contiene una sección de patrones relacionados con la arquitectura.
- Artículos disponibles a través de la web sobre patrones de arquitectura (como las arquitecturas J2EE), en Sun, IBM, y otros sitios web.
- Artículos disponibles a través del web sobre arquitectura en el Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*) de la Universidad Carnegie Mellon, que desde hace mucho tiempo es un centro de investigación sobre arquitectura (www.sei.cmu.edu).

DISEÑO DE MÁS REALIZACIONES DE CASOS DE USO CON OBJETOS Y PATRONES

*En dos ocasiones me han preguntado (miembros del Parlamento),
“Por favor, Mr. Babbage, si introduce en la máquina cifras incorrectas,
¿obtendrá la respuesta correcta?” No soy capaz de comprender exactamente el
tipo de confusión de ideas que podría provocar tal pregunta.*

Charles Babbage

Objetivos

- Aplicar los patrones GRASP y GoF en el diseño.
-

Introducción

Este capítulo estudia algunos diseños parciales para la iteración actual, manejando re-
quisitos tales como el mantenimiento de los servicios ante fallos mediante servicios lo-
cales, la gestión de dispositivos de PDV y la autorización de pagos.

33.1. Mantenimiento de los servicios ante los fallos mediante servicios locales; rendimiento con el almacenamiento local

Uno de los requisitos de NuevaEra es un cierto grado de recuperación ante el fallo
(*failover*) del servicio remoto, como una base de datos de productos no disponible
(temporalmente).

El acceso a la información de los productos es el primer caso que se utiliza para es-
tudiar la estrategia de recuperación y mantenimiento del servicio ante un fallo. Poste-

riormente, se explora el acceso al servicio de contabilidad, que tiene una solución ligeramente distinta.

Revisemos parte del memorándum técnico:

Memorándum Técnico

Asunto: Fiabilidad—Recuperación de fallos en los servicios remotos

Resumen de la solución: Ubicación transparente utilizando un servicio de búsqueda, mantenimiento del servicio ante los fallos pasando de remoto a local, y replicación parcial en el servicio local.

Factores

- Recuperación robusta de los fallos en los servicios remotos (ej. calculador de impuestos, inventario).
- Recuperación robusta de los fallos en la base de datos de productos (ej. descripciones y precios) remota.

Solución

Conseguir variaciones protegidas con respecto a la ubicación de los servicios utilizando un Adaptador creado en una *FactoriaDeServicios*. Donde sea posible, ofrecer implementaciones locales de servicios remotos, normalmente con comportamiento simplificado o restringido. Por ejemplo, el calculador de impuestos local utilizará porcentajes de impuestos constantes. La base de datos local con información de los productos será una pequeña caché de los productos más comunes. Se almacenarán las actualizaciones del inventario y se remitirán cuando se restablezca la conexión.

Véase también el memorándum técnico de *Adaptabilidad—Servicios de terceras partes* para conocer los aspectos de adaptabilidad de esta solución, porque las implementaciones de los servicios remotos variarán en cada instalación.

Para satisfacer los escenarios de calidad de reconexión con los servicios remotos, utilizar para los servicios objetos Proxy inteligentes, que en cada llamada a un servicio comprueban si se puede reactivar el servicio remoto y lo redireccionan a éste cuando es posible.

Motivación

¡Las tiendas realmente no quieren dejar de vender! Por tanto, si el PDV NuevaEra ofrece este nivel de fiabilidad y recuperación, será un producto muy atractivo, ya que ninguno de nuestros competidores proporciona esta capacidad.

Antes de solucionar los aspectos de recuperación y de mantenimiento del servicio ante los fallos, observe que tanto por razones de rendimiento como para mejorar la recuperación cuando falla la base de datos remota, el arquitecto (yo) ha recomendado una caché local (que persiste de manera fiable en el disco duro local en un simple fichero) de objetos *EspecificacionDelProducto*. Por tanto, siempre se debería buscar en la caché local por si hubiera un “acierto de caché” antes de intentar un acceso remoto.

Esto se puede conseguir de manera elegante con nuestro diseño existente del adaptador y la factoría:

1. La *FactoriaDeServicios* siempre devolverá un adaptador a un servicio de información de producto local.
2. El “adaptador” de los productos locales realmente no es un adaptador a otro componente. Implementará él mismo las responsabilidades del servicio local.

3. El valor inicial del servicio local es una referencia a un segundo adaptador del verdadero servicio de productos remoto.
4. Si el servicio local encuentra el dato en su caché, lo devuelve; en otro caso, remite la petición al adaptador del servicio externo.

Obsérvese que existen dos niveles de caché del lado del cliente:

1. El objeto *CatalogoDeProductos* en memoria mantendrá una colección en memoria (como un *HashMap* de Java) de algunos (por ejemplo, 1.000) objetos *EspecificacionDelProducto* que se han obtenido a través del servicio de información de productos. Se puede ajustar el tamaño de esta colección en función de la disponibilidad de memoria local.
2. El servicio de productos local mantendrá una caché persistente más grande (en el disco duro) que mantiene cierta cantidad de información de productos (como 1 o 100 MB de espacio en ficheros). De nuevo, se puede ajustar dependiendo de la configuración local. Esta caché persistente es importante para la tolerancia a fallos, de manera que incluso si la aplicación del PDV cae y se pierde la caché de memoria del objeto *CatalogoDeProductos*, permanece la caché persistente.

Este diseño no afecta al código existente —el nuevo objeto de servicio local se inserta sin afectar al diseño del objeto *CatalogoDeProductos* (que colabora con el servicio de productos)—.

Hasta el momento, no se han introducido nuevos patrones; se han utilizado un Adaptador y una Factoría.

La Figura 33.1 ilustra los tipos del diseño y la Figura 33.2 ilustra la inicialización.

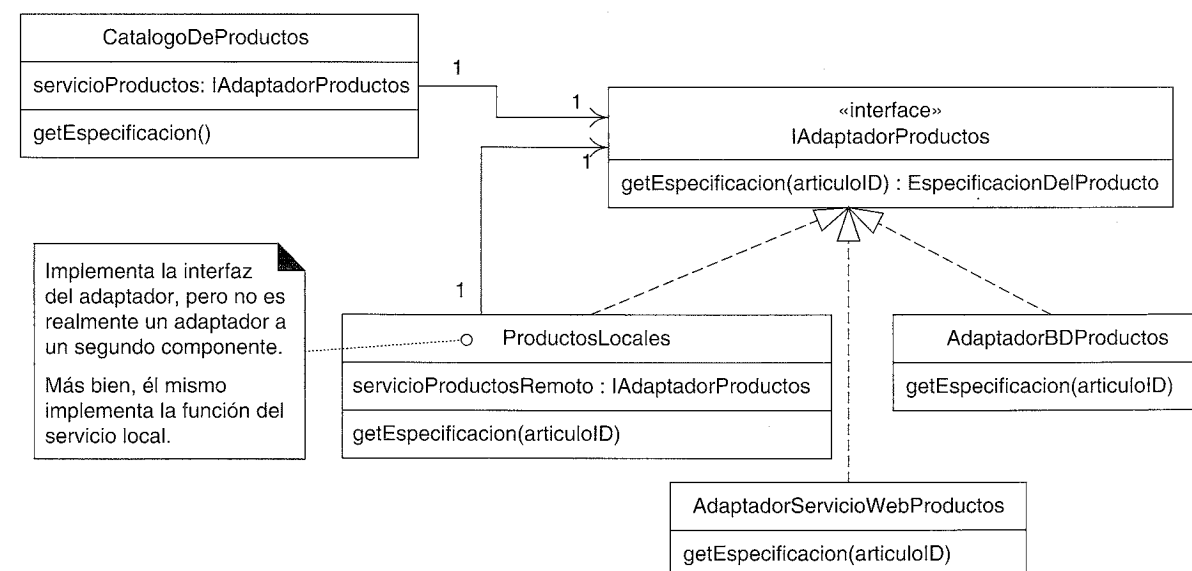


Figura 33.1. Adaptadores para la información de productos.

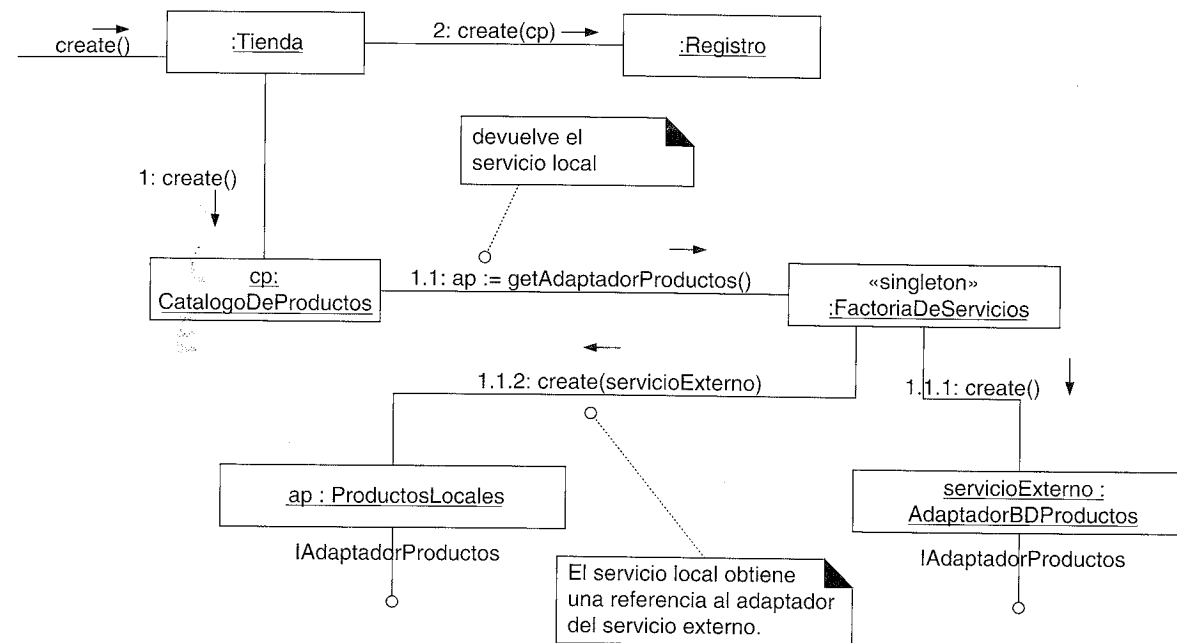


Figura 33.2. Inicialización del servicio de información de productos.

La Figura 33.3 muestra la colaboración inicial desde el catálogo al servicio de productos.

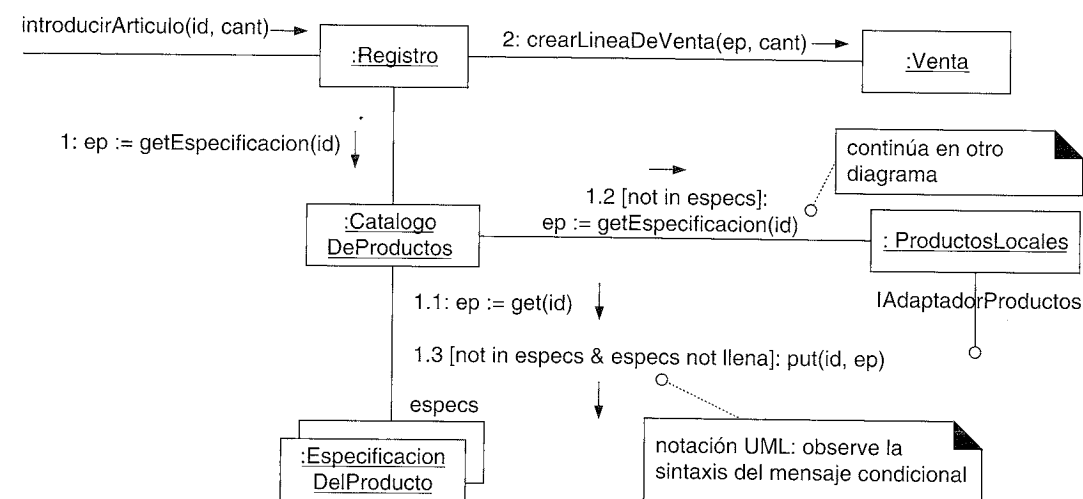


Figura 33.3. Comienzo de la colaboración con el servicio de productos.

Si el servicio de productos local no tiene el producto en su caché, colabora con el adaptador del servicio externo, como se muestra en la Figura 33.4. Nótese que el servicio de productos local almacena los objetos *EspecificacionDelProducto* como verdaderos objetos serializados.

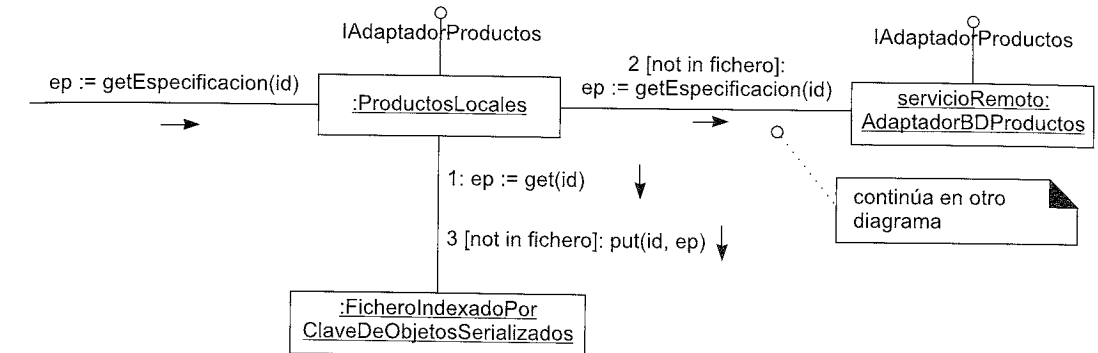


Figura 33.4. Continuación de la colaboración para obtener la información del producto.

Si se cambió el servicio externo de una base de datos a un nuevo servicio Web, sólo es necesario cambiar la configuración de la factoría del servicio remoto. Véase la Figura 33.5.

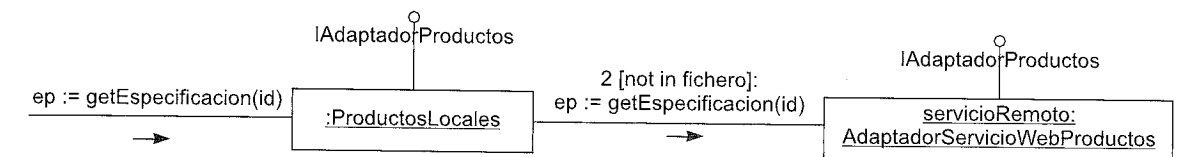


Figura 33.5. Los nuevos servicios externos no afectan al diseño.

Continuando con el caso de la colaboración con el *AdaptadorBDProductos*, interactuará con un subsistema de persistencia que tiene que establecer la correspondencia objeto-relacional (*mapping O-R*) (ver Figura 33.6).

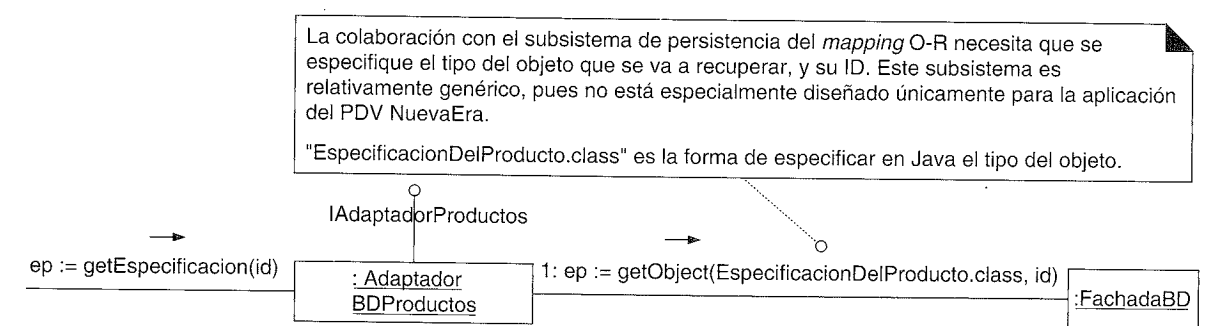


Figura 33.6. Colaboración con el subsistema de persistencia.

Estrategias de almacenamiento en caché

Considere las alternativas para cargar en memoria la caché del *CatalogoDeProductos* y la caché basada en ficheros de *ProductosLocales*: un enfoque es la inicialización perezosa, en el que las cachés se cargan lentamente cuando se recupera la información del producto externo; otro enfoque es la inicialización impaciente, en el que las cachés se cargan du-

rante el caso de uso *PonerEnMarcha*. Si el diseñador no está seguro del enfoque a utilizar y quiere experimentar con las alternativas, una familia de diferentes objetos *Estrategia-Cache* basada en el patrón Estrategia puede solucionar el problema de manera elegante.

Caché antigua

Puesto que los precios de los productos cambian rápidamente, y quizás al antojo del encargado de la tienda, almacenar en la caché el precio de los productos crea un problema —la caché contiene datos antiguos—; esto es siempre una preocupación cuando se replican los datos. Una solución es añadir una operación al servicio remoto que responda con los cambios actuales a lo largo de un día; el objeto *ProductosLocales* la consulta cada n minutos y actualiza su caché.

Hilos en UML

Si el objeto *ProductosLocales* va a solucionar el problema de la cache con datos antiguos cada n minutos, un enfoque de diseño es hacerlo un **objeto activo** que posea un hilo de control. El hilo dormirá durante n minutos, se despertará, el objeto obtendrá los datos, y

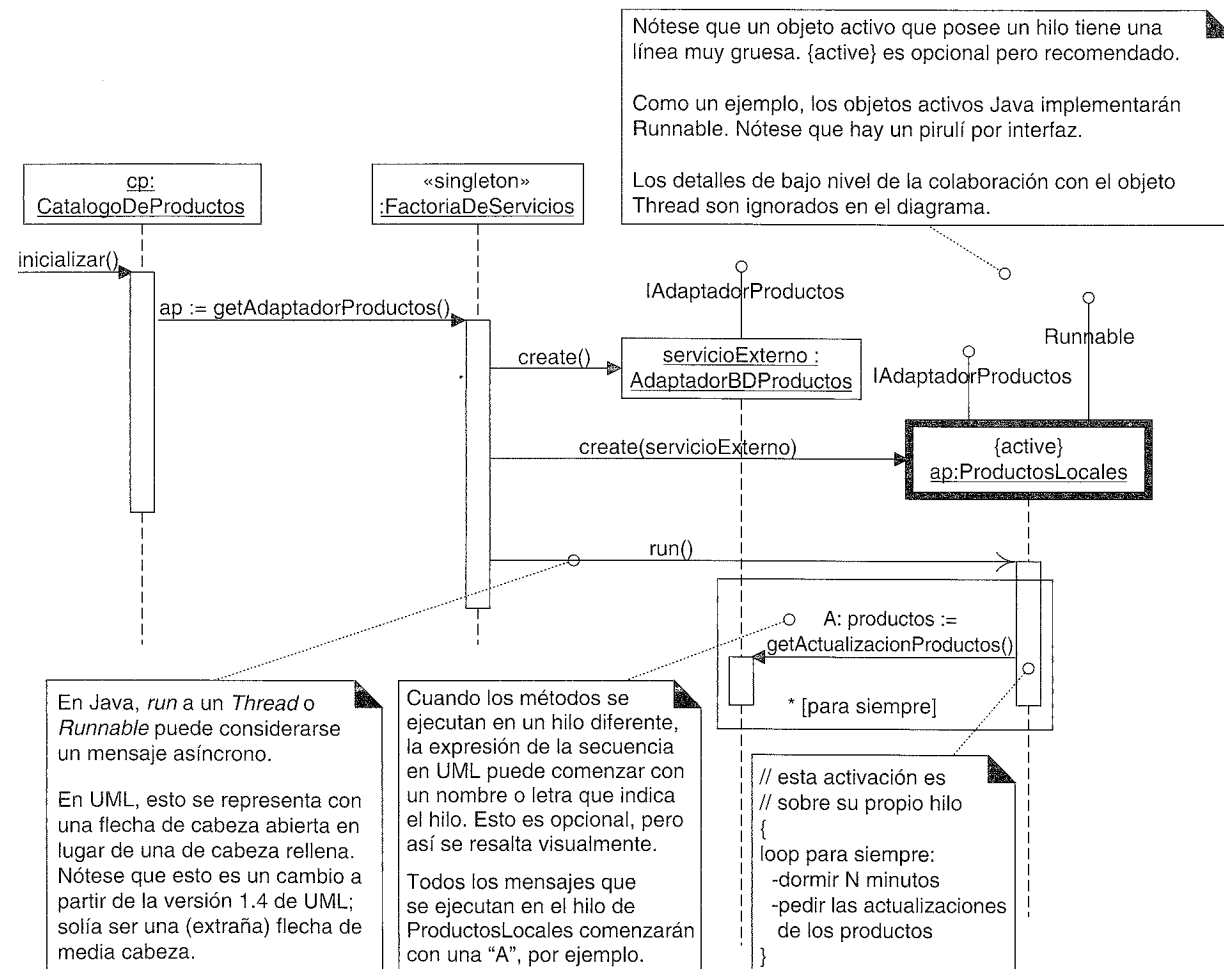


Figura 33.7. Hilos y mensajes asíncronos en UML.

el hilo volverá a dormirse. UML proporciona la notación para representar hilos y las llamadas asíncronas, como se muestra en la Figura 33.7.

En un diagrama de interacción, una instancia de un objeto activo podría etiquetarse con la propiedad {active}. En un diagrama de clases, una clase de objetos activos (una **clase activa**) que posee su propio hilo se puede estereotipar con «thread». Véase la Figura 33.8.

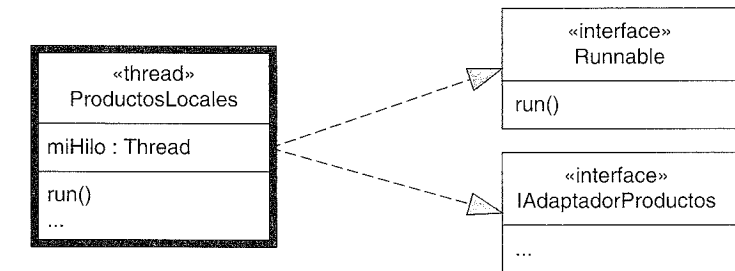


Figura 33.8. Notación para las clases activas.

33.2. Manejo de fallos

El diseño anterior proporciona una solución para el almacenamiento en la caché del lado del cliente de los objetos *EspecificacionDelProducto* en un fichero persistente, para mejorar el rendimiento, y también para proporcionar al menos una solución parcial a la que recurrir si no se puede acceder al servicio de productos externo. Quizás se almacenen en la caché 10.000 productos en el fichero local, que podría satisfacer la mayoría de las peticiones de información de los productos incluso cuando falla el servicio externo.

¿Qué hacemos en el caso de que no se encuentre en la caché y falla el acceso al servicio de productos externo? Suponga que las personas involucradas nos piden que creamos una solución que indique al cajero que introduzca manualmente el precio y la descripción, o cancele la entrada de la línea de venta.

Éste es un ejemplo de una condición de error o fallo, y se utilizará como contexto para describir algunos patrones generales que se ocupan del manejo de los fallos y excepciones. El manejo de excepciones y errores es un tema amplio, y esta introducción únicamente se centrará en algunos patrones específicos para el contexto del caso de estudio. En primer lugar, algo de terminología:

- **Defecto:** el origen último o causa del mal comportamiento.
 - El programador escribe mal el nombre de la base de datos.
- **Error:** una manifestación del defecto en el sistema en ejecución. Los errores se detectan (o no).
 - Cuando estamos invocando al servicio de nombres para obtener una referencia a la base de datos (con el nombre mal escrito), se señala un error.
- **Fallo:** la denegación de un servicio causada por un error.
 - El subsistema de Productos (y el PDV NuevaEra) falla al proporcionar un servicio de información de productos.

Lanzamiento de excepciones

Un enfoque directo para señalar el fallo que se está produciendo es lanzar una excepción.

Las excepciones son apropiadas especialmente cuando estamos tratando con fallos de los recursos (disco, memoria, acceso a la red o a una base de datos y otros servicios externos).

Se lanzará una excepción desde el interior del subsistema de persistencia (realmente, es muy probable que a partir de algo como una implementación JDBC de Java), donde se detecte en primer lugar un fallo al utilizar la base de datos de productos externa. La excepción desenrollará la pila de llamadas hacia atrás hasta un punto apropiado donde se pueda manejar¹.

Suponga que la excepción original (utilizando Java como ejemplo) es *java.sql.SQLException*. ¿Debería propagarse una *SQLException* todo el camino hacia arriba hasta llegar a la capa de presentación? No. No está en el nivel correcto de abstracción. Esto nos lleva a un patrón común para el manejo de excepciones:

Patrón: Convertir Excepciones [Brown01]

En un subsistema, evite la emisión de excepciones de bajo nivel que proceden de los subsistemas inferiores o servicios. Más bien, convierta una excepción de bajo nivel en una que sea significativa para el nivel del subsistema. La excepción de más alto nivel normalmente encapsula la excepción de un nivel inferior, y añade información, para hacer que la excepción sea más significativa de acuerdo con el contexto de los niveles más altos.

Esto es una guía, no una regla absoluta.

Aquí "Excepción" se utiliza en el sentido vernáculo de algo que se puede lanzar; en Java, el equivalente es un *Throwable*.

También conocido como Abstracción de Excepción [Renzel97].

Por ejemplo, el subsistema de persistencia captura una *SQLException* concreta, y (asumiendo que no puede manejarla²) lanza una nueva *BDNoDisponibleException*, que contiene la *SQLException*. Nótese que el *AdaptadorBDProductos* es como una fachada sobre un subsistema lógico para la información de los productos. Por tanto, el *AdaptadorBDProductos* de más alto nivel (como representante de un subsistema lógico) captura la *BDNoDisponibleException* de un nivel inferior y (asumiendo que no puede manejarla) lanza una nueva *InfoProductoNoDisponibleException*, que encapsula la *BDNoDisponibleException*.

¹ No se trata el manejo de excepciones comprobadas frente a las no comprobadas, ya que no es soportado en todos los lenguajes OO conocidos—C++, C# y Smalltalk, por ejemplo.

² Resolver una excepción cerca del nivel donde se produce es un objetivo loable pero difícil, puesto que los requisitos acerca de la manera de manejar un error con frecuencia son específicos de la aplicación.

Considere los nombres de estas excepciones: ¿por qué decir *BDNoDisponibleException* en lugar de *SubsistemaPersistenciaException*? Existe un patrón para esto:

Patrón: Nombre El Problema No El Lanzador [Grosso00]

¿Cómo llamar a una excepción? Asigne un nombre que describa por qué se va a lanzar la excepción, no quién la lanza. La ventaja es que facilita al programador la comprensión del problema, y resalta la similitud fundamental de muchas clases de excepciones (de una manera que no ocurre nombrando al que la lanza).

Excepciones en UML

Éste es un momento adecuado para introducir la notación de UML para el lanzamiento³ y captura de excepciones.

Notación UML: UML tiene una sintaxis "por defecto" para las operaciones. Pero no incluye una solución oficial para mostrar las excepciones que lanza una operación. Existen al menos tres soluciones:

1. UML permite que se utilice para las operaciones la sintaxis de cualquier otro lenguaje, como Java. Además, algunas herramientas CASE UML permiten mostrar por pantalla las operaciones explícitamente en la sintaxis de Java. De este modo,

Object get(Clave, Class) throws BDNoDisponibleException, FatalException

2. La sintaxis por defecto permite que el último elemento sea un "string de propiedades" (*property string*). Esto es una lista arbitraria de pares propiedad+valor, como {autor=Craig, niños=(Hannah, Haley),...}. Así,

put(Object, id){ throws=(BDNoDisponibleException, FatalException)}

3. Algunas herramientas CASE UML le permiten a uno especificar (en un cuadro de diálogo especial) las excepciones que lanza una operación.

Las excepciones que se capturan se modelan como un tipo de operación que maneja una señal.

Las excepciones que se lanzan se pueden listar en otro compartimento etiquetado "excepciones"

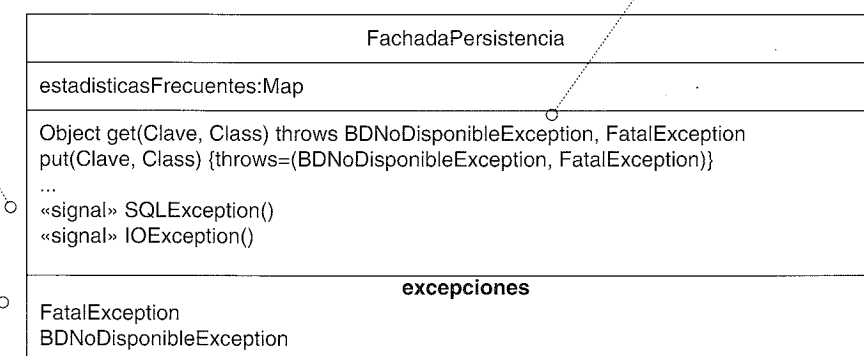


Figura 33.9. Excepciones capturadas y lanzadas por una clase.

³ Oficialmente en UML, uno *envía* una excepción, pero *lanza* es un término adecuado y más familiar.

Dos preguntas típicas sobre la notación UML son:

1. En un diagrama de clases, ¿cómo representamos las excepciones que captura y lanza una clase?
2. En un diagrama de interacción, ¿cómo representamos el lanzamiento de una excepción?

La Figura 33.9 presenta la notación para un diagrama de clases.

En UML, una *Exception* es una especialización de una *Signal*, que es la especificación de una comunicación asíncrona entre objetos. Esto significa que en un diagrama de interacción, las excepciones se representan como **mensajes asíncronos**⁴.

La Figura 33.10 muestra la notación, utilizando como ejemplo la descripción anterior de *SQLException* que se transforma en una *BDNoDisponibleException*.

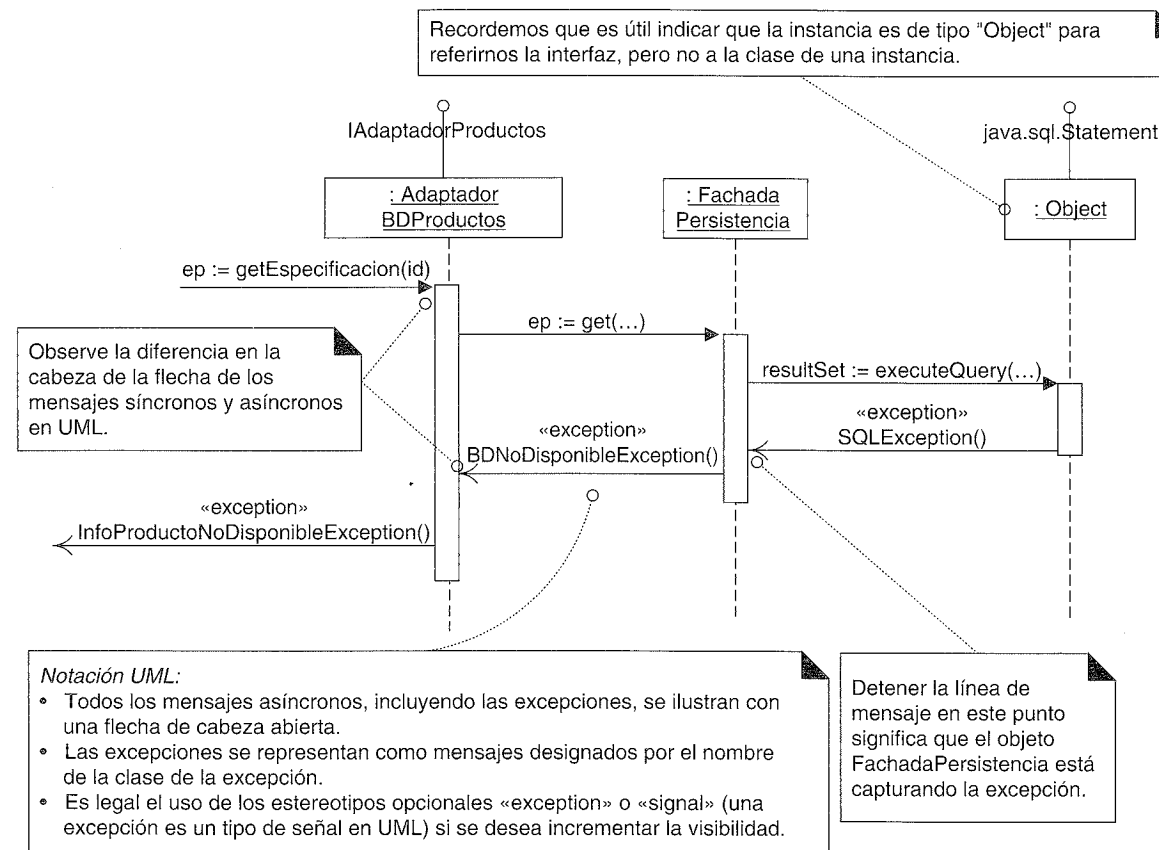


Figura 33.10. Excepciones en un diagrama de interacción.

En resumen, existe notación UML para representar las excepciones. Sin embargo, pocas veces se utiliza.

⁴ Nótese que a partir de la versión 1.4 de UML, cambió la notación para los mensajes asíncronos de una flecha de media cabeza a una de cabeza abierta.

No es que recomendemos evitar las consideraciones iniciales sobre el manejo de excepciones. Más bien lo contrario: al nivel de arquitectura es necesario establecer pronto los patrones básicos, políticas y colaboraciones para el manejo de las excepciones, puesto que es difícil insertar el manejo de las excepciones como una ocurrencia tardía. Sin embargo, muchos desarrolladores consideran que el diseño a bajo nivel del manejo de excepciones concretas se decide de manera más apropiada durante la programación o mediante descripciones de diseño menos detalladas, en lugar de mediante diagramas UML detallados.

Manejo de errores

Se ha considerado una parte del diseño: el lanzamiento de excepciones, en cuanto a la conversión, asignación de nombres y la manera de representarlas. La otra parte es el manejo de una excepción.

Dos patrones que se pueden aplicar en éste y en la mayoría de los casos son:

Patrón: Registro Centralizado de los Errores [Renzel97]

Utilice un objeto central de registro de errores con acceso mediante el patrón Singleton y notifíquelo todas las excepciones. Si se trata de un sistema distribuido, cada singleton local colaborará con un registro de error central. Beneficios:

- Informes consistentes.
- Definición flexible de la información de salida y el formato.

También conocido como Registrador de Diagnóstico (*Diagnostic Logger*) [Harrison98].

Es un patrón sencillo. El segundo es:

Patrón: Diálogo de Error [Renzel97]

Para notificar los errores a los usuarios utilice un objeto que no pertenezca a la interfaz de usuario, independiente de la aplicación y al que se accede siguiendo el patrón Singleton. Actúa como envoltorio (*wrapper*) de uno o más objetos de "diálogo" de la UI (tales como diálogo modal de la UI, consola de texto, emitir un sonido, o generador de voz) y delega la notificación del error a los objetos del UI. De esta manera, la salida podría ir tanto a un diálogo de la GUI como a un generador de voz. También informará de la excepción al registro centralizado de errores. Una Factoría que lee los parámetros del sistema creará los objetos apropiados de la UI. Beneficios:

- Variaciones Protegidas con respecto a los cambios en el mecanismo de salida.
- Estilo consistente de los informes de error; por ejemplo, todas las ventanas de la GUI pueden invocar a este singleton para mostrar el diálogo de error.
- Control centralizado de la estrategia común para la notificación de los errores.
- Ganancia de rendimiento pequeña; si se utiliza un recurso "caro" como un diálogo de la GUI, es fácil ocultarlo y almacenarlo en caché para volver a utilizarlo después, en lugar de volver a crear un diálogo por cada error.

¿Debería un objeto de la UI (por ejemplo, *FrameProcesarVenta*) manejar un error capturando la excepción e informando al usuario? Para aplicaciones con pocas ventanas,

y caminos de navegación entre ventanas simples y estables, este diseño directo es bueno. Esto se cumple actualmente para la aplicación NuevaEra.

Tenga presente sin embargo, que esto sitúa algo de la “lógica de la aplicación” relacionada con el manejo de los errores en la capa de presentación (GUI). El manejo de errores está relacionado con la notificación a los usuarios, luego es lógica, pero es una tendencia que hay que tener en cuenta. No es inherentemente un problema de UI simples con pocas oportunidades de reemplazar la UI, sino que es un punto de fragilidad. Por ejemplo, suponga que un equipo quiere sustituir la UI Swing de Java por el framework de GUI Java MicroView de IBM para ordenadores portátiles. Ahora existe algo de la lógica de la aplicación en la versión Swing que se tiene que identificar y replicar en la versión MicroView. Hasta cierto punto, esto es inevitable en las sustituciones de UI; pero se agravará cuanto más lógica de la aplicación se migre hacia arriba. En general, cuantas más responsabilidades de la lógica de la aplicación que no tiene que ver con el UI se migren a la capa de presentación, mayor es la probabilidad de que se produzcan quebraderos de cabeza relacionados con el diseño o el mantenimiento.

Para sistemas con muchas ventanas y caminos de navegación complejos (quizás incluso cambiantes), existen otras soluciones. Por ejemplo, se puede insertar una capa de aplicación de uno o más controladores entre las capas de presentación y del dominio.

Además, se puede insertar un objeto “mediador en el manejo de la vista” [GHJV95, BMRSS96] que es responsable de mantener una referencia a todas las ventanas abiertas, y conocer las transiciones entre ventanas, dado algún evento E1 (como un error).

Este mediador es de manera abstracta una máquina de estados que encapsula los estados (la ventana que se muestra) y las transiciones entre los estados, en base a los eventos. Podría leer el modelo de transición de estados (ventanas) de un fichero externo, de manera que los caminos de navegación pueden ser dirigidos por los datos (no es necesario ningún cambio en el código fuente). También puede cerrar todas las ventanas de la aplicación, o disponerlas de algún modo o minimizarlas, puesto que tiene una referencia a todas las ventanas.

En este diseño, podría diseñarse un controlador de la capa de aplicación con una referencia a este mediador para el manejo de la vista (por ello, el controlador de la aplicación se acopla “hacia arriba” con la capa de presentación). El controlador de la aplicación podría capturar la excepción y colaborar con el mediador para el manejo de la vista para producir la notificación (basado en el patrón Diálogo de Error). De esta manera, el controlador de la aplicación está involucrado con flujos de trabajo de la aplicación, y se deja fuera de las ventanas algo del manejo lógico de errores.

El diseño detallado del control de la UI y de la navegación quedan fuera del alcance de esta introducción, y el simple diseño de la ventana que captura la excepción será suficiente. Un diseño que utiliza un Diálogo de Error se muestra en la Figura 33.11.

33.3. Mantenimiento de los servicios ante los fallos mediante un Proxy (GoF)

El mantenimiento de los servicios ante los fallos (*failover*) mediante un servicio local para la información de los productos se consiguió insertando el servicio local delante del servicio externo; el servicio local siempre se prueba primero. Sin embargo, este diseño

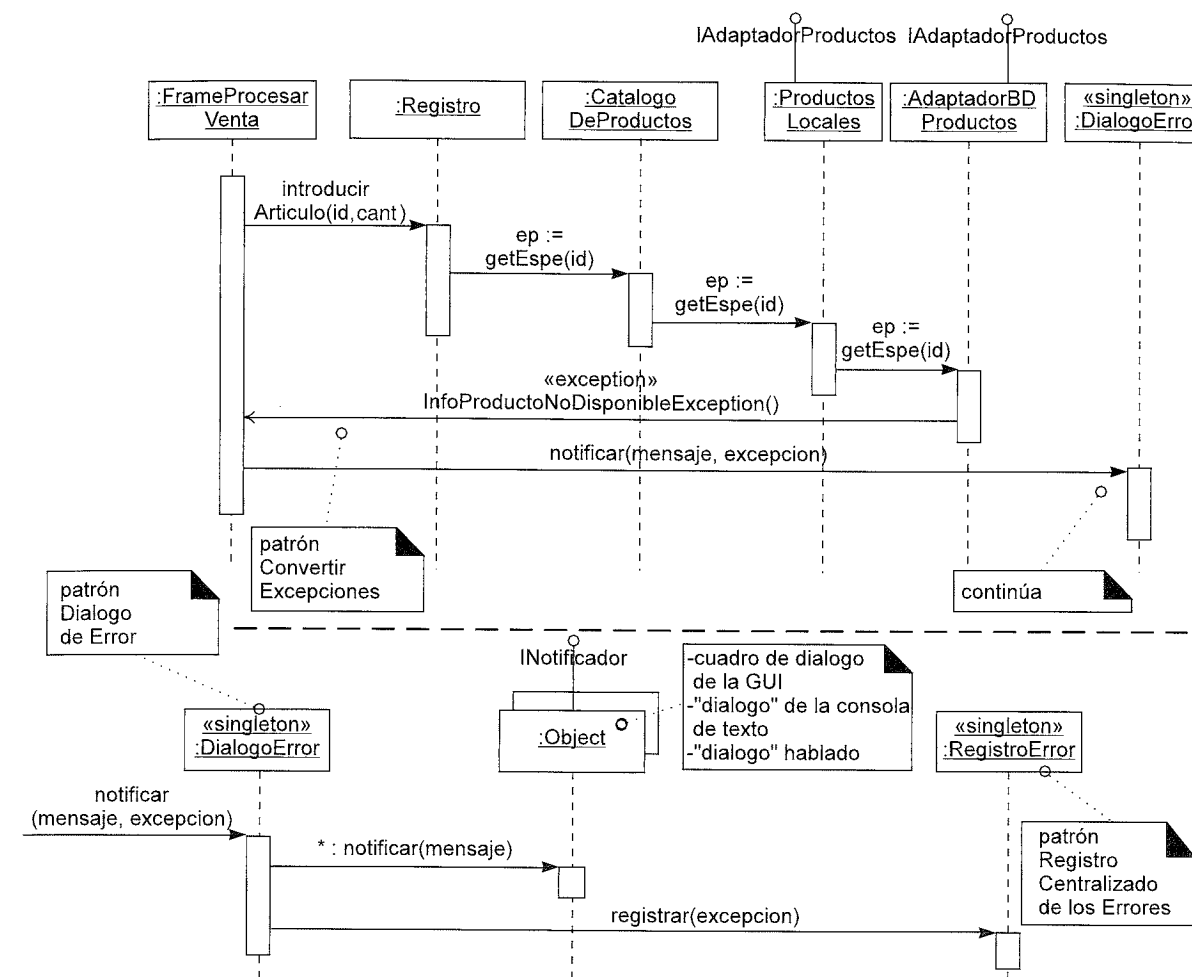


Figura 33.11. Manejo de la excepción.

no es apropiado para todos los servicios; algunas veces se debería probar primero el servicio externo, y la versión local en segundo lugar. Por ejemplo, considere el registro de las ventas en el servicio de contabilidad. El negocio quiere que se registren lo antes posible, para controlar en tiempo real la actividad de la tienda y el registro.

En este caso, otro patrón GoF puede solucionar el problema: Proxy⁵. Proxy es un patrón sencillo, y se utiliza ampliamente su variante **Proxy Remoto**. Por ejemplo, en RMI de Java y en CORBA, un objeto local del lado del cliente (denominado “*stub*”) se invoca para que acceda a los servicios de un objeto remoto. El stub del lado del cliente es un proxy local, o un representante o sustituto del objeto remoto.

Este ejemplo de uso del Proxy en NuevaEra no es la variante de Proxy Remoto, sino la variante **Proxy de Redirección** (*Redirection Proxy*) (también conocido como **Proxy de Mantenimiento de Servicio**, *Failover Proxy*).

⁵ *N. del T.* No se ha traducido por el mismo motivo que en el caso del patrón Singleton.

Independientemente de la variante, la estructura de un Proxy es siempre la misma; las variantes tienen que ver con lo que hace el proxy una vez que se le ha llamado.

Un proxy es simplemente un objeto que implementa la misma interfaz que el objeto al que se quiere acceder en realidad, manteniendo una referencia al sujeto real, y suele controlar el acceso a él. La Figura 33.12 muestra la estructura general.

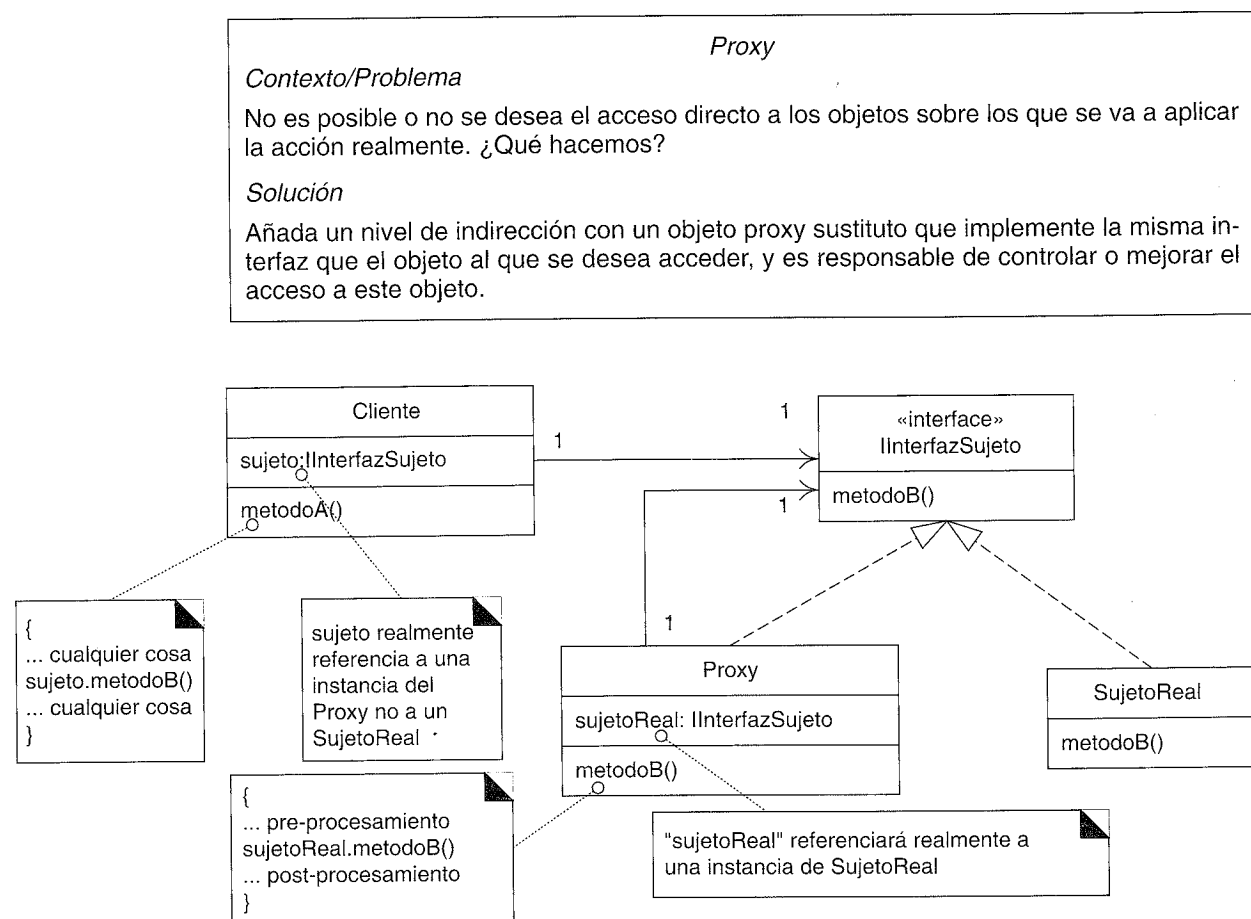


Figura 33.12. Estructura general del patrón Proxy.

Aplicado al caso de estudio de NuevaEra para el acceso al servicio de contabilidad externo, se utiliza un proxy de redirección como sigue:

1. Enviar el mensaje *anotarVenta* al proxy de redirección, tratándolo como si se pensase que es el servicio de contabilidad externo real.
2. Si falla el proxy de redirección al establecer el contacto con el servicio externo (mediante su adaptador), entonces redireccionar el mensaje *anotarVenta* a un servicio local, que almacena localmente las ventas para remitirlas al servicio de contabilidad, cuando esté disponible.

La Figura 33.13 ilustra un diagrama de clases de los elementos interesantes.

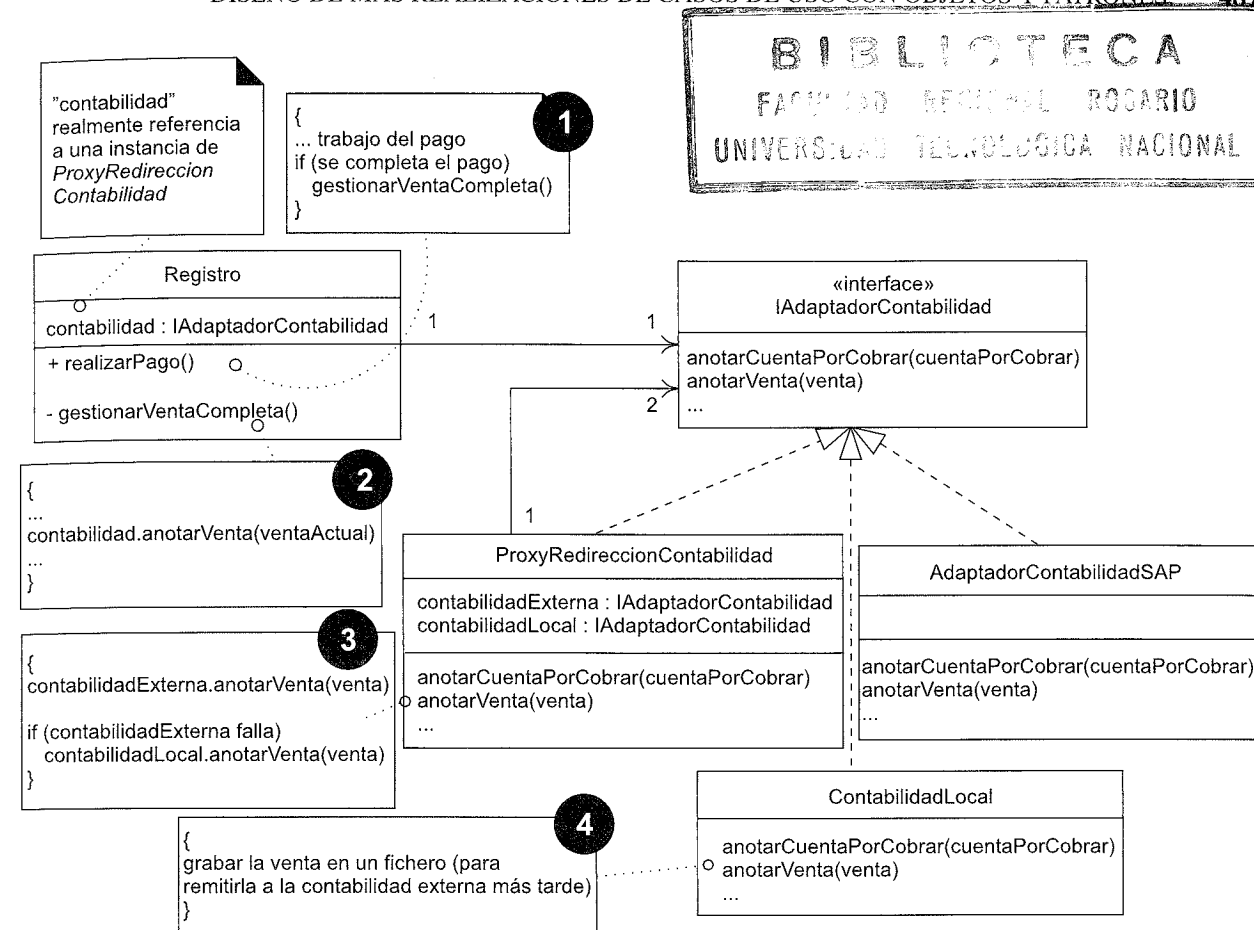


Figura 33.13. Uso en NuevaEra de un proxy de redirección.

Notación UML:

- Para evitar crear un diagrama de interacción que muestre el comportamiento dinámico, obsérvese cómo este diagrama estático utiliza la numeración para mostrar la secuencia de interacción. Normalmente es preferible un diagrama de interacción, pero se presenta este estilo para ilustrar un estilo alternativo.
- Obsérvese los marcadores de visibilidad pública y privada (+, -) junto a los métodos del *Registro*. Si no se indican, significa que no se especifica en lugar de asumir un valor por defecto público o privado. Sin embargo, de acuerdo con la práctica habitual, la mayoría de las personas que leen diagramas UML (y las herramientas CASE que generan código) interpretan la visibilidad no especificada como privada para los atributos y pública para los métodos. Sin embargo, en este diagrama, en particular quiero transmitir el hecho de que *realizarPago* es público, y en cambio, *gestionarVentaCompleta* es privado. El ruido visual y la sobrecarga de información son siempre una preocupación en la comunicación, luego, es conveniente hacer uso de las interpretaciones convencionales para mantener los diagramas simples.

Resumiendo, un proxy es un objeto externo que envuelve un objeto interno, y ambos implementan la misma interfaz. Un objeto cliente (como un *Registro*) no sabe que refe-

rencia a un proxy —se diseña de manera que piense que está colaborando con el sujeto real (por ejemplo, el *AdaptadorContabilidadSAP*)—. El Proxy intercepta las llamadas para mejorar el acceso al sujeto real, en este caso redireccionando la operación a un servicio local (*ContabilidadLocal*) si no está accesible el servicio externo.

33.4. Diseño para los requisitos no funcionales o de calidad

Antes de pasar a la siguiente sección, observe que el trabajo de diseño realizado hasta este punto del capítulo no está relacionado con la lógica del negocio, sino con los requisitos no funcionales o de calidad relacionados con la fiabilidad y recuperación.

Curiosamente —y es un punto clave en la arquitectura del software— es habitual que los diseños den forma antes a los temas, patrones y estructuras de la arquitectura del software de gran escala para resolver los requisitos no funcionales o de calidad, en lugar de a la lógica del negocio básica.

33.5. Acceso a los dispositivos físicos externos con adaptadores; comprar vs. construir

Otro requisito de esta iteración es interactuar con dispositivos físicos que componen un terminal de PDV, como abrir el cajón de caja, entregar el cambio del dispensador de monedas y capturar la firma de un dispositivo para las firmas digitales.

El PDV NuevaEra debe trabajar con una variedad de equipamiento de PDV, que incluye el que vende IBM, Epson, NCR, Fujitsu, etcétera.

Afortunadamente, el arquitecto de software ha investigado algo, y ha descubierto que ahora existe un estándar industrial, UnifiedPOS⁶ (www.nrf-arts.org), que define las interfaces orientadas a objetos estándar (en el sentido de UML) para todos los dispositivos comunes del PDV. Además, existe el JavaPOS (www.javapos.com) —la correspondencia en Java del UnifiedPOS—.

Por tanto, en el Documento de la Arquitectura del Software, el arquitecto añade un memorándum técnico para comunicar esta alternativa significativa para la arquitectura:

Memorándum Técnico

Asunto: Fiabilidad—Control de los dispositivos hardware del PDV

Resumen de la solución: Utilizar el software de Java de los fabricantes de dispositivos que se ajusta a las interfaces estándares de JavaPOS.

Factores

- Controlar correctamente los dispositivos
- Coste de comprar vs. construir y mantener

⁶ N. del T. POS es acrónimo de *Point-Of-Sale* (Punto-De-Venta), por tanto en esta sección aparece tanto POS como PDV.

Solución

UnifiedPOS (www.nrf-arts.org) define un modelo de interfaces UML que es un estándar industrial para los dispositivos de PDV. JavaPOS (www.javapos.com) es un estándar industrial que establece la correspondencia del UnifiedPOS a Java. Los fabricantes de dispositivos de PDV (ej. IBM, NCR) venden las implementaciones Java de estas interfaces que controlan sus dispositivos.

Comprarlos en lugar de construirlos.

Utilizar una Factoría que lea una propiedad del sistema para cargar un conjunto de clases de IBM o NCR (etc.) y devuelva instancias basadas en sus interfaces.

Motivación

Basado en una encuesta informal, creemos que funcionan bien, y los fabricantes tienen un proceso de actualización regular para mejorarlos. Es difícil conseguir la experiencia y otros recursos para escribirlos nosotros mismos.

Alternativas consideradas

Escribirlos nosotros mismos —difícil y arriesgado—.

La Figura 33.14 muestra algunas de las interfaces, que se han añadido como otro paquete de la capa del dominio en nuestro Modelo de Diseño.

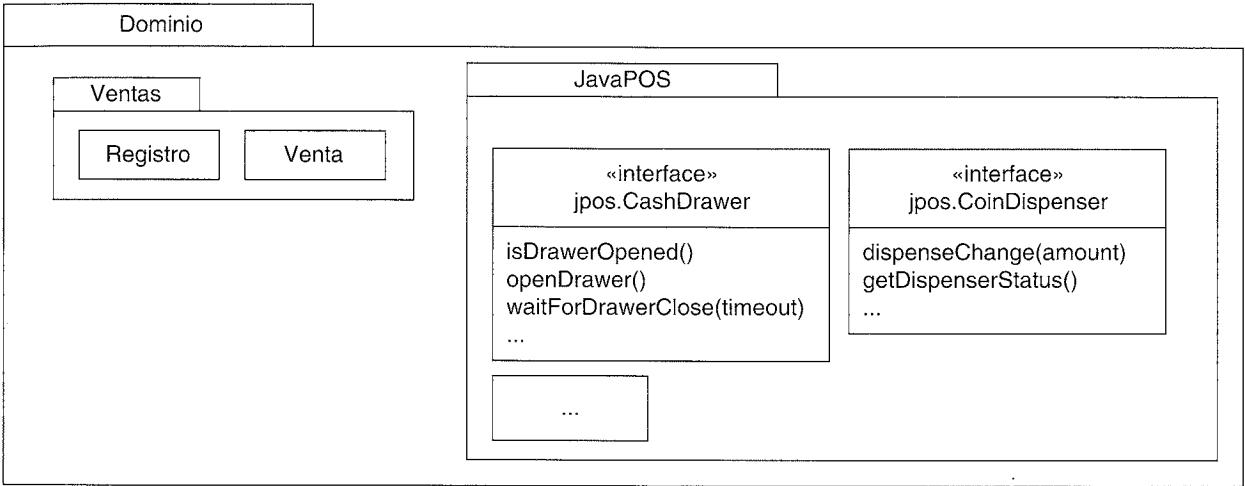


Figura 33.14. Interfaces JavaPOS estándares.

Asuma que los fabricantes importantes de los equipos de PDV ahora suministran las implementaciones JavaPOS. Por ejemplo, si compramos un terminal de PDV IBM con un cajón de caja, dispensador de monedas, etcétera, IBM también nos puede proporcionar las clases Java que implementan las interfaces JavaPOS, y que controlan los dispositivos físicos.

En consecuencia, esta parte de la arquitectura se resuelve comprando componentes software, en lugar de construyéndolos. Fomentar el uso de componentes existentes es una de las buenas prácticas del UP.

¿Cómo funcionan? A bajo nivel, un dispositivo físico tiene un controlador de dispositivo para las operaciones del sistema operativo subyacente. Una clase Java (por ejemplo, una que implemente *jpos.CashDrawer*) utiliza JNI (*Java Native Interface*) para hacer las llamadas a estos controladores de dispositivos.

Esas clases de Java adaptan los controladores de dispositivos de bajo nivel a las interfaces de JavaPOS, y así se pueden caracterizar como objetos Adaptador en el sentido del patrón GoF. También se les puede llamar objetos Proxy —objetos proxy locales que controlan o mejoran el acceso a los dispositivos físicos—.

No es raro que seamos capaces de clasificar un diseño en función de múltiples patrones.

33.6. Factoría Abstracta (GoF) para familias de objetos relacionados

Se comprarán a los fabricantes las implementaciones JavaPOS. Por ejemplo⁷:

```
//controladores IBM
com.ibm.pos.jpos.CashDrawer (implements jpos.CashDrawer)
com.ibm.pos.jpos.CoinDispenser (implements jpos.CoinDispenser)
...
//Controladores NCR
com.ncr.posdrivers.CashDrawer (implements jpos.CashDrawer)
com.ncr.posdrivers.CoinDispenser (implements jpos.CoinDispenser)
...
```

Ahora, ¿cómo diseñamos la aplicación de PDV NuevaEra para utilizar los controladores Java de IBM, si es que se utiliza el hardware de IBM, los controladores NCR si fueran los apropiados, etcétera?

Nótese que hay familias de clases (*CashDrawer*+*CoinDispenser*+...) que es necesario que se creen, y cada familia implementa las mismas interfaces.

Para esta situación, existe un patrón GoF que se utiliza comúnmente: Factoría Abstracta (*Abstract Factory*).

Factoría Abstracta (Abstract Factory)

Contexto/Problema

¿Cómo crear familias de clases relacionadas que implementen una interfaz común?

Solución

Defina una interfaz para la factoría (la factoría abstracta). Defina una clase factoría concreta para cada familia de cosas que hay que crear. Opcionalmente, defina una clase abstracta que implemente la interfaz factoría y proporcione servicios comunes a las factorías concretas que la extienden.

⁷ Estos son nombres de paquetes ficticios.

La Figura 33.15 ilustra la idea básica, que se mejora en la siguiente sección.

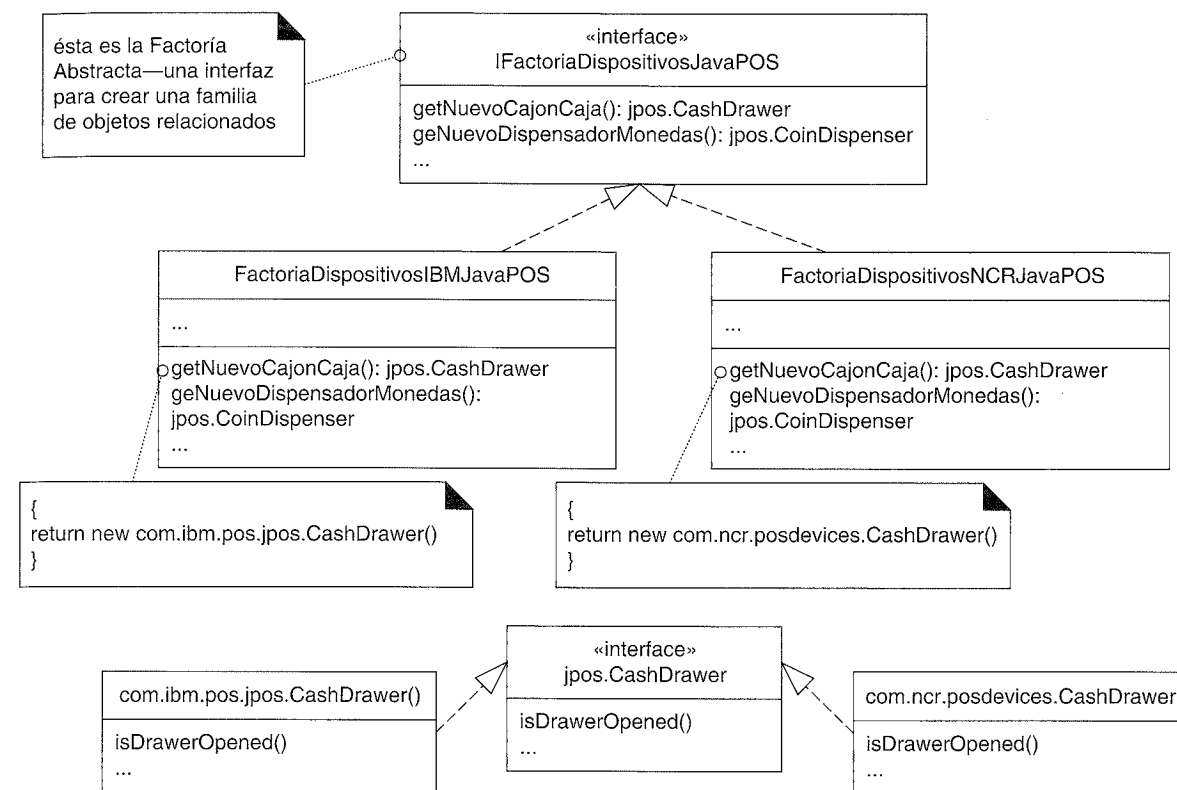


Figura 33.15. Una factoría abstracta básica.

Factoría Abstracta mediante una clase abstracta

Una variación típica de la Factoría Abstracta es crear una clase abstracta factoría a la que se accede utilizando el patrón Singleton; lee una propiedad del sistema para decidir cuál de sus subclases factoría crear, y entonces devuelve la instancia de la subclase apropiada. Esto se utiliza, por ejemplo, en las librerías de Java con la clase *java.awt.Toolkit*, que es una clase abstracta que representa una factoría abstracta para la creación de familias de elementos gráficos del GUI para diferentes sistemas operativos y subsistemas de GUI.

La ventaja de este enfoque es que soluciona este problema: ¿cómo sabe la aplicación qué factoría abstracta utilizar? ¿*FactoriaDispositivosIBMJavaPOS*? ¿*FactoriaDispositivosNCRJavaPOS*?

El siguiente refinamiento soluciona este problema. La Figura 33.16 ilustra la solución.

Con esta clase abstracta como factoría y el método *getInstancia* del patrón Singleton, los objetos pueden colaborar con la superclase abstracta, y obtener una referencia a una de las instancias de sus subclases. Por ejemplo, considere la declaración:

```
cajonCaja=FactoriaDispositivosJavaPOS.getInstancia().getNuevoCajonCaja();
```

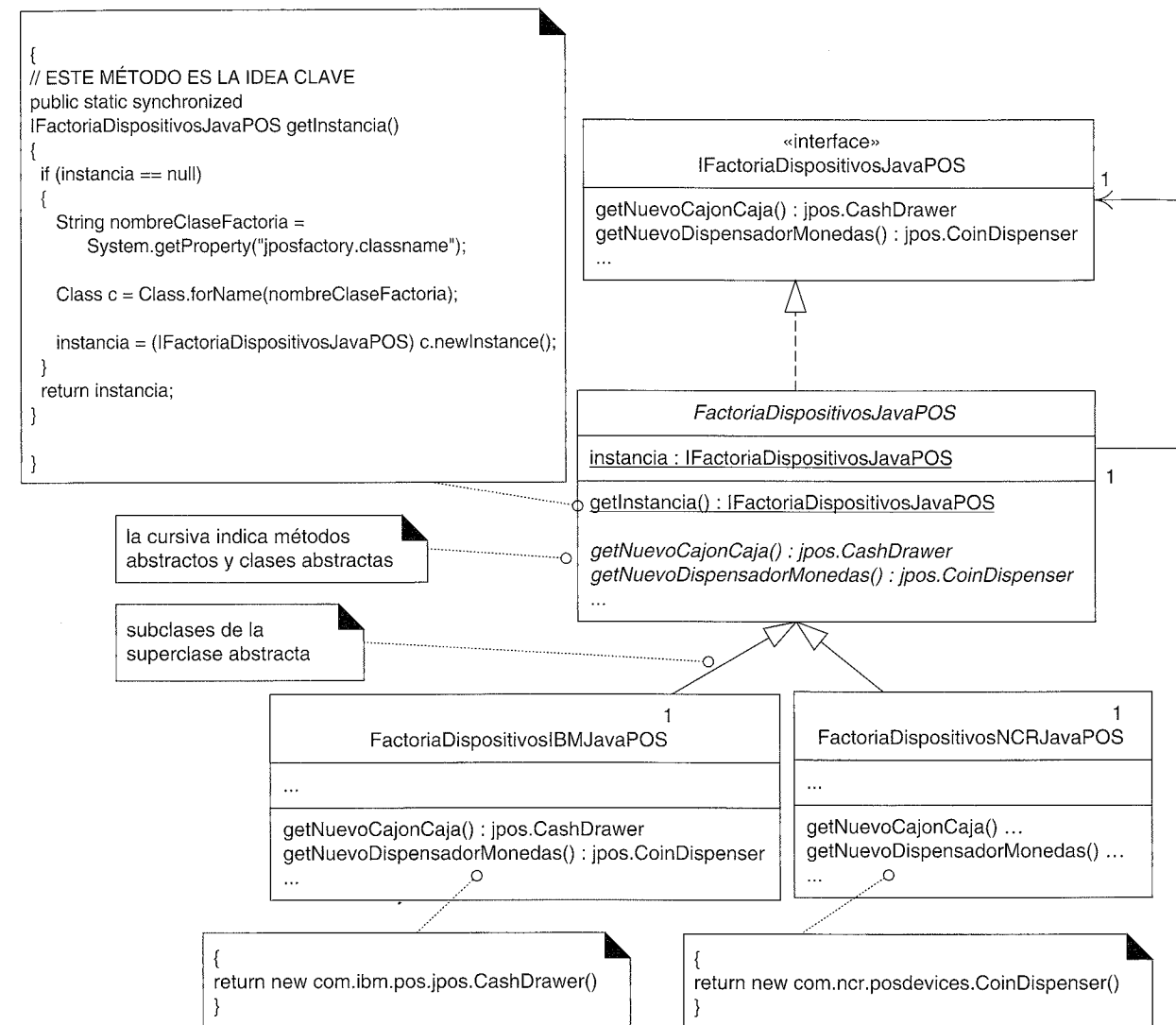


Figura 33.16. Factoría Abstracta mediante una clase abstracta.

La expresión *FactoriaDispositivosJavaPOS.getInstancia()* devolverá una instancia de *FactoriaDispositivosIBMJavaPOS* o *FactoriaDispositivosNCRJavaPOS*, dependiendo de la propiedad del sistema que se lea. Obsérvese que cambiando la propiedad del sistema externa "jposfactory.classname" (que es el nombre de la clase como un String) en un fichero de propiedades, el sistema NuevaEra utilizará una familia diferente de controladores JavaPOS. Se consiguen Variaciones Protegidas con respecto a los cambios de factoría con un diseño de programación dirigido por los datos (leyendo un fichero de propiedades) y reflexivo, utilizando la expresión *c.newInstance()*.

La interacción con la factoría tendrá lugar en un *Registro*. De acuerdo con el objetivo de salto en la representación bajo, es razonable que el *Registro* software (cuyo nom-

bre sugiere el terminal de PDV global) mantenga una referencia a los dispositivos como el *CajonCaja*. Por ejemplo:

```

class Registro
{
    private jpos.CashDrawer cajonCaja;
    private jpos.CoinDispenser dispensadorMonedas;

    public Registro()
    {
        cajonCaja =
            FactoriaDispositivosJavaPos.getInstancia().getNuevoCajonCaja();
        //...
    }
    //...
}

```

33.7. Gestión de pagos con Polimorfismo y Hacerlo Yo Mismo

Una de las formas habituales de aplicación del polimorfismo (y el Experto en Información) es en el contexto de lo que Peter Coad denomina la estrategia o patrón "Hacerlo Yo Mismo" [Coad95]. Es decir:

Hacerlo Yo Mismo

"Yo (un objeto software) hago las cosas que hace normalmente el objeto actual del que yo soy una abstracción." [Coad95]

Éste es el estilo de diseño orientado a objetos clásico: los objetos *Circulo* se dibujan ellos mismos, los objetos *Cuadrado* se dibujan ellos mismos, los objetos *Texto* hacen ellos mismos la comprobación ortográfica, y así sucesivamente.

Nótese que el hecho de que un objeto *Texto* compruebe él mismo la ortografía es un ejemplo del Experto en Información: *El objeto que tiene la información relacionada con el trabajo lo hace* (un *Diccionario* también es un candidato, según el Experto).

Hacerlo Yo Mismo y el Experto en Información normalmente nos llevan a la misma elección.

Análogamente, observe que el hecho de que el *Circulo* y el *Cuadrado* se dibujen ellos mismos son ejemplos de Polimorfismo: *Cuando alternativas relacionadas varían según el tipo, asigne la responsabilidad utilizando operaciones polimórficas a los tipos para los que varía el comportamiento*.

Hacerlo Yo Mismo y el Polimorfismo normalmente nos llevan a la misma elección.

No obstante, como se estudió en la discusión acerca de la Fabricación Pura, con frecuencia está contraindicado debido a problemas con el acoplamiento y la cohesión, y en

lugar de eso, un diseñador utiliza fabricaciones puras como estrategias, factorías y otras por el estilo.

A pesar de todo, cuando es apropiado, Hacerlo Yo Mismo es atractivo en parte porque favorece un salto en la representación bajo. El diseño para la gestión de pagos se llevará a cabo con los patrones Hacerlo Yo Mismo y Polimorfismo.

Uno de los requisitos de esta iteración es gestionar múltiples tipos de pagos, que esencialmente significa gestionar los pasos de autorización y contabilidad. Diferentes tipos de pagos se autorizan de distintas formas:

- Los pagos a crédito y débito se autorizan con un servicio de autorización externo. Ambos requieren que se registre una entrada en las cuentas por cobrar —dinero que debe la institución financiera que hace la autorización—.
- En algunas tiendas (es una tendencia en algunos países) se autorizan los pagos en efectivo utilizando un analizador especial de billetes unido al terminal de PDV que comprueba si el dinero es falso. Otras tiendas no lo hacen.
- En algunas tiendas se autorizan los pagos con cheque utilizando un servicio de autorización informatizado. Otras tiendas no autorizan los cheques.

Los *PagosACredito* se autorizan de una forma; los *PagosConCheque* se autorizan de otra. Éste es un caso clásico de Polimorfismo.

De esta manera, como se muestra en la Figura 33.17, cada subclase de *Pago* tiene su propio método *autorizar*.

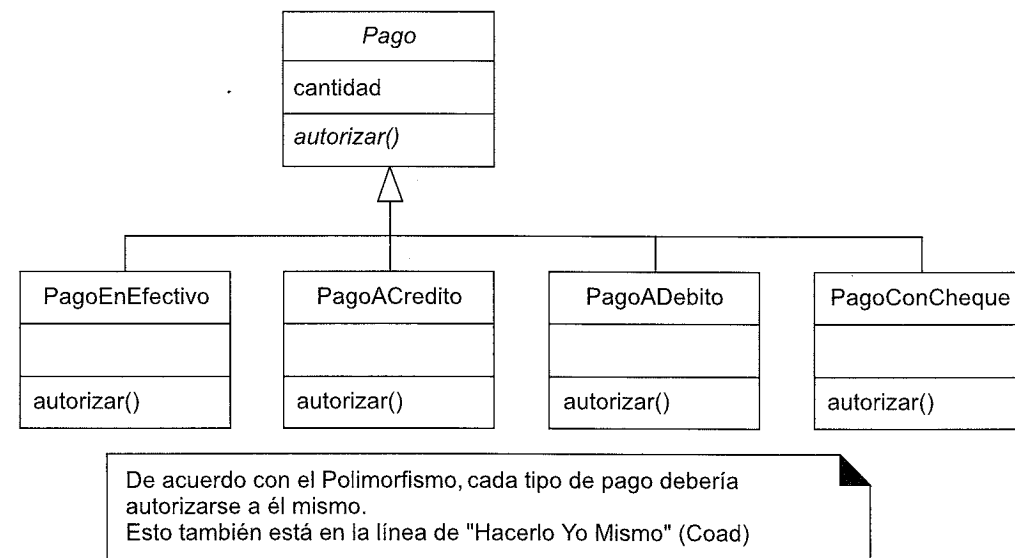


Figura 33.17. Polimorfismo clásico con múltiples métodos *autorizar*.

Por ejemplo, como se ilustra en las Figuras 33.18 y 33.19, una *Venta* instancia un *PagoACredito* o un *PagoConCheque* y le pide que se autorice él mismo.

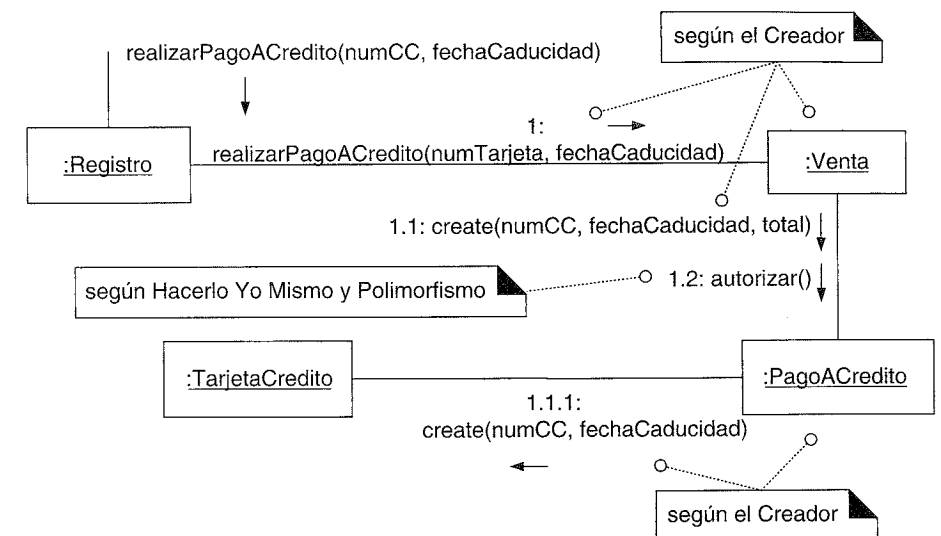


Figura 33.18. Creación de un *PagoACredito*.

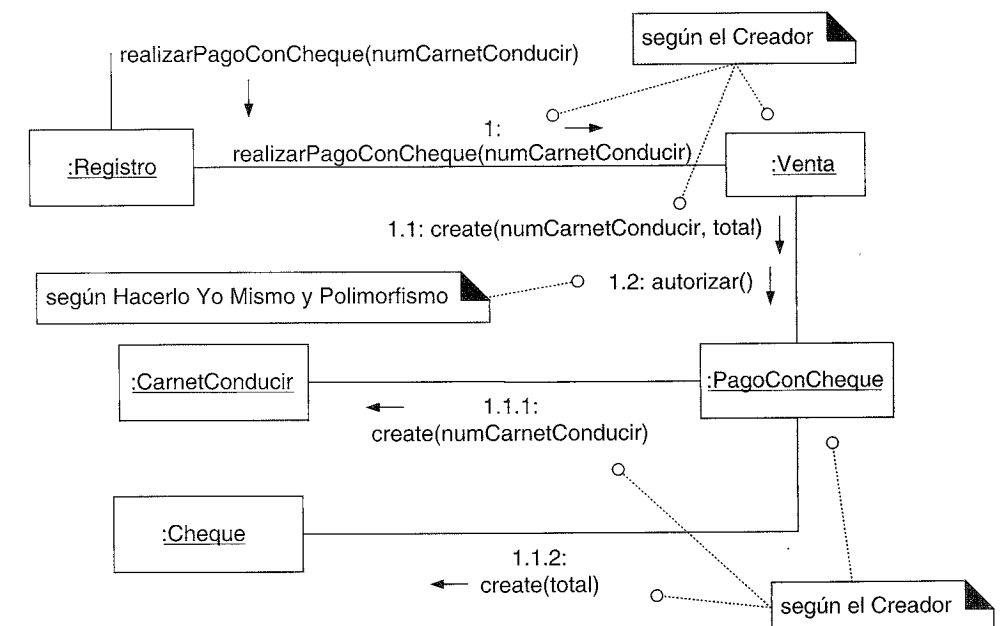


Figura 33.19. Creación de un *PagoConCheque*.

¿Clases pequeñas?

Considere la creación de los objetos software *TarjetaCredito*, *CarnetConducir* y *Cheque*. Nuestro primer impulso podría ser simplemente registrar los datos que contienen en sus clases de pago relacionadas, y eliminar tales clases pequeñas. Sin embargo, normalmente utilizarlas es una estrategia más beneficiosa; a menudo terminan proporcionando com-

portamiento útil y acaban siendo reutilizables. Por ejemplo, la *TajetaCredito* es un Experto natural en decirnos el tipo de compañía de crédito (Visa, MasterCard, etcétera). Este comportamiento resultará necesario para nuestra aplicación.

Autorización de pago a crédito

El sistema se debe comunicar con un servicio externo de autorización de crédito, y ya hemos creado la base del diseño para dar soporte a esto basada en adaptadores.

Información del dominio sobre el pago a crédito relevante

Establezcamos el contexto para el diseño que vamos a realizar a continuación:

- Los sistemas PDV se conectan físicamente con los servicios externos de autorización de varias formas, entre las que se encuentran líneas telefónicas (que se deben marcar) y conexiones permanentes de Internet de banda ancha.
- Se utilizan diferentes protocolos del nivel de la aplicación y formatos de datos asociados, como Transacciones Electrónicas Seguras (SET, *Secure Electronic Transaction*). Podrían hacerse populares otros nuevos, como XMLPay.
- La autorización de pagos se puede ver como una operación síncrona ordinaria: un hilo de ejecución del PDV se bloquea, esperando una respuesta del servicio remoto (dentro de los límites de un periodo de tiempo de espera).
- Todos los protocolos de autorización de pago conllevan el envío de identificadores que identifican de manera única a la tienda (con un "ID de comerciante"), y al terminal del PDV (con un "ID del terminal"). Una respuesta incluye un código de aprobación o denegación, y un ID de transacción único.
- Una tienda podría utilizar diferentes servicios de autorización externos para distintos tipos de tarjetas de crédito (uno para la VISA y otro para la MasterCard). Para cada servicio, la tienda tiene un ID de comerciante distinto.
- El tipo de la compañía de crédito se puede deducir a partir del número de la tarjeta. Por ejemplo, los números que comienzan por 5 son MasterCard; los números que empiezan por 4 son Visa.
- Las implementaciones de los adaptadores protegerán a las capas superiores del sistema contra todas estas variaciones en la autorización de los pagos. Cada adaptador es responsable de asegurar que la transacción de solicitud de la autorización tiene el formato adecuado, y de la colaboración con el servicio externo. Como se discutió en la iteración anterior, la *FactoriaDeServicios* es responsable de enviar la implementación de *IAdaptadorServicioAutorizacionCredito* adecuada.

Un escenario de diseño

La Figura 33.20 comienza la presentación de un diseño con anotaciones que satisfacen estos detalles y requisitos. Los mensajes tienen notas aclaratorias para mostrar el razonamiento.

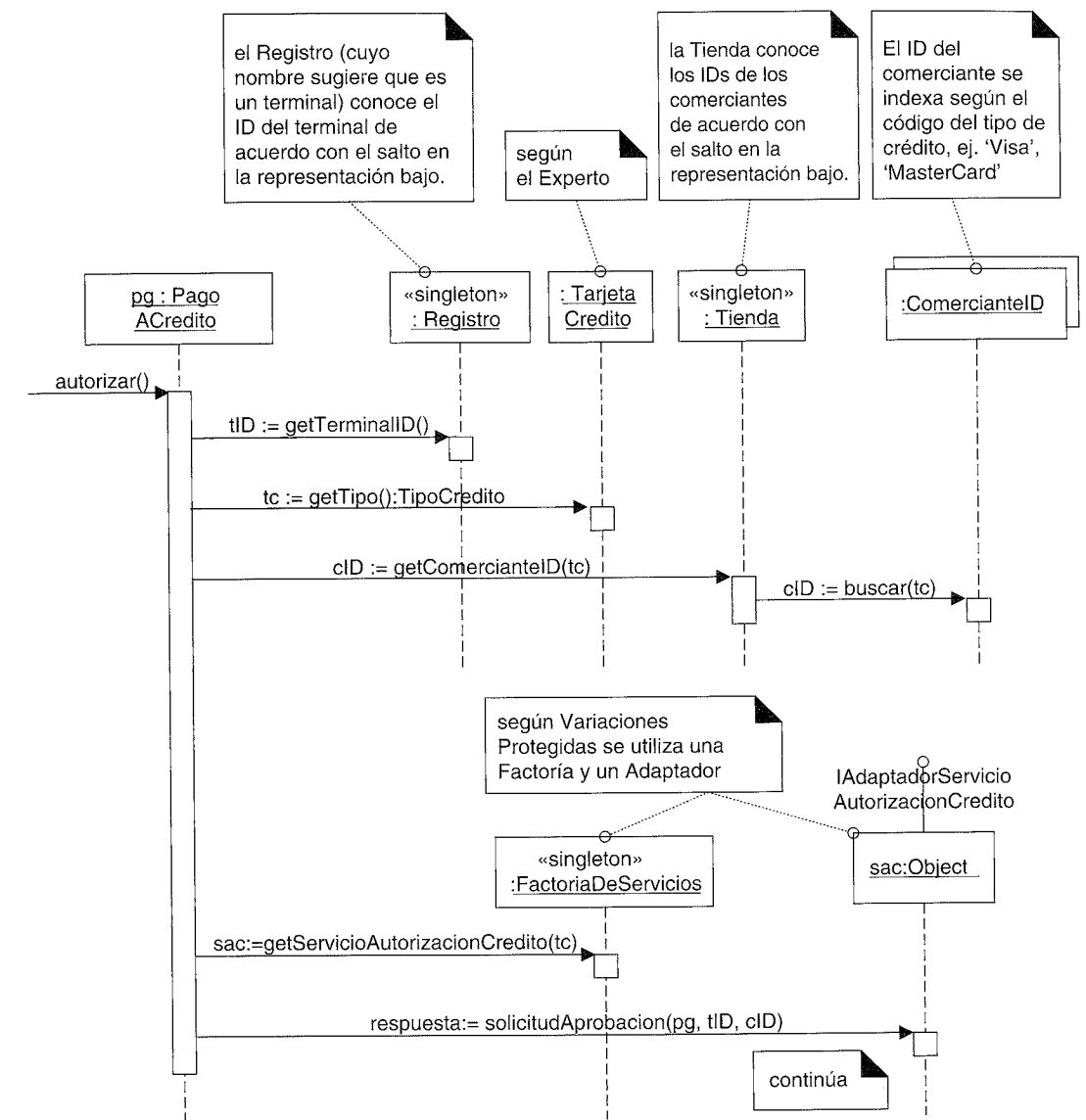


Figura 33.20. Gestión de un pago a crédito.

Una vez que se encuentra el *IAdaptadorServicioAutorizacionCredito* correcto, se le otorga la responsabilidad de completar la autorización, como se muestra en la Figura 33.21.

En el momento que se obtiene la respuesta del *PagoACredito* (al que se le ha dado la responsabilidad de gestionar su finalización de acuerdo con el Polimorfismo y Hacerlo Yo Mismo), asumiendo que se aprueba, completa sus tareas, como se muestra en la Figura 33.22.

Notación UML: Obsérvese en este diagrama de secuencia que se apilaron algunos objetos. Esto es legal, aunque pocas herramientas CASE lo soportan. Es de utilidad para incluirlo en un libro o documento, donde el ancho de página está limitado.

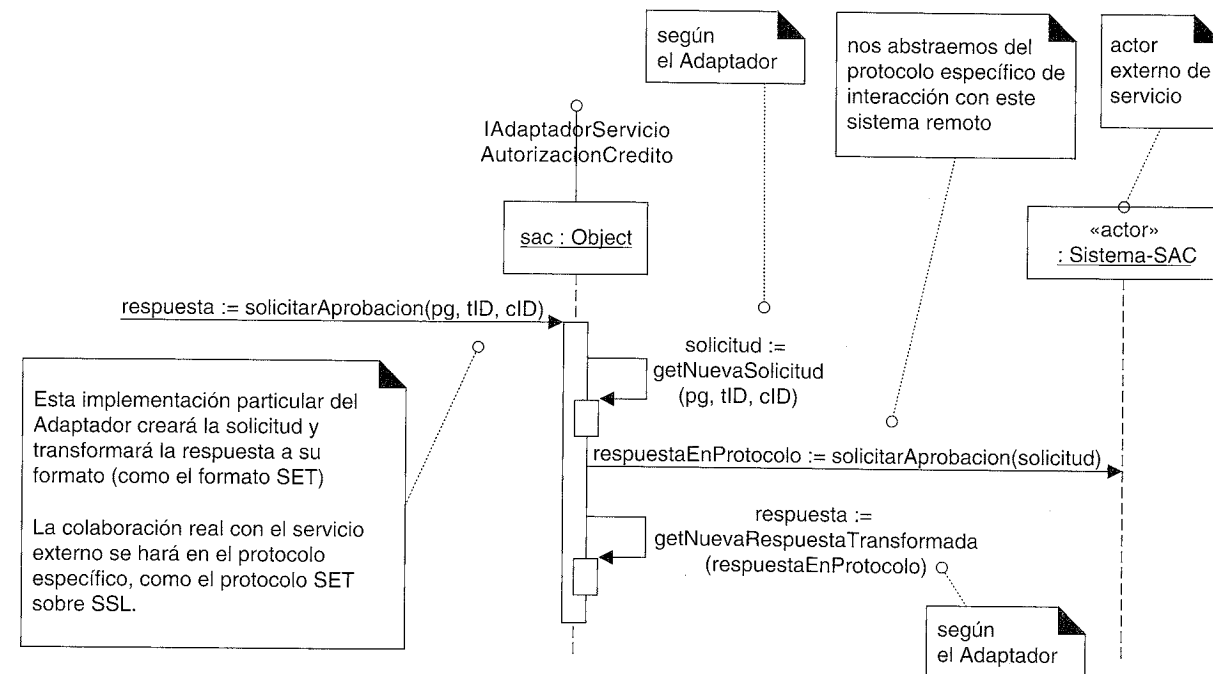


Figura 33.21. Finalización de la autorización.

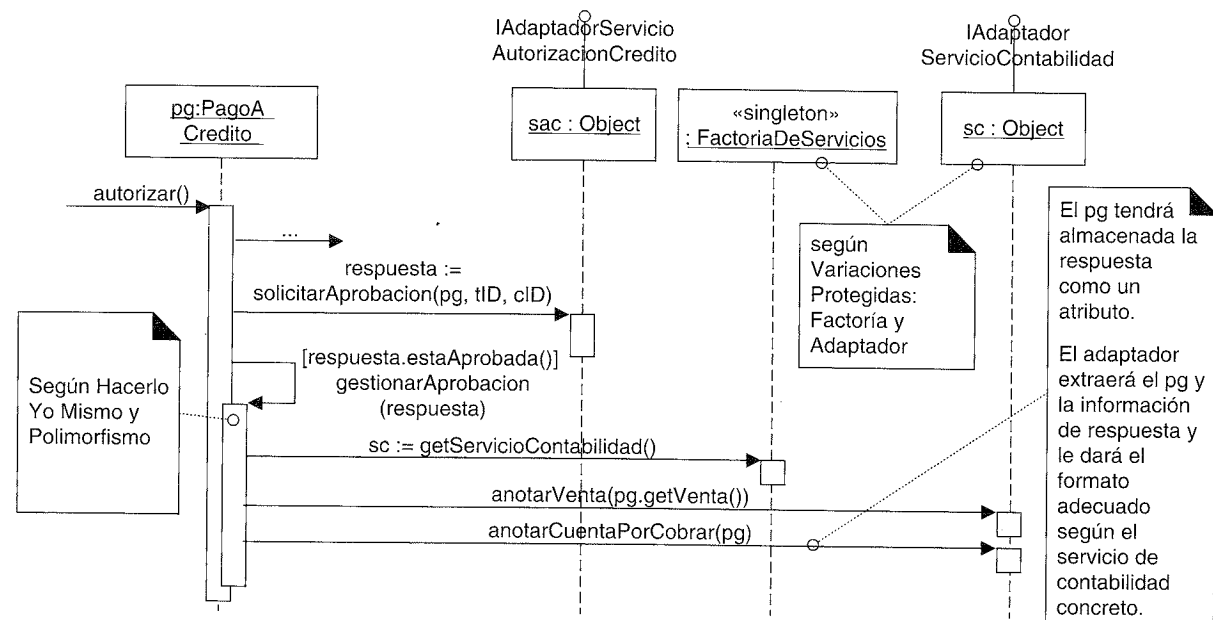


Figura 33.22. Finalización de un pago a crédito aprobado.

33.8. Conclusión

Lo importante de este caso de estudio no era mostrar la solución correcta —no existe una única solución que sea la mejor—, y estoy seguro de que los lectores pueden me-

jorar lo que he propuesto. Espero sinceramente haber demostrado que se puede llevar a cabo el diseño de objetos basado en un razonamiento que siga principios básicos como bajo acoplamiento y la aplicación de patrones, en lugar de ser un proceso misterioso.

Advertencia: Patron-itis

Esta presentación ha utilizado los patrones de diseño GoF en muchos puntos, lo cual es una de las cosas importantes del caso de estudio como ayuda al aprendizaje. Pero, ha habido informes de diseñadores preocupados excesivamente por introducir los patrones de una manera forzada en un frenesí creativo de patron-itis. La conclusión que podemos extraer de esto es que es necesario estudiar los patrones en múltiples ejemplos para digerirlos bien. Un método de aprendizaje extendido es formar un grupo de estudio a la hora de la comida o después del trabajo en el que los participantes comparten las formas que han visto o podrían ver de la aplicación de los patrones, y discutir una sección de un libro sobre patrones.

DISEÑO DE UN FRAMEWORK DE PERSISTENCIA CON PATRONES

*Le temps est un grand professeur, mais
malheureusement il tue tous ses élèves*
(El tiempo es un gran profesor, pero desgraciadamente mata a todos sus alumnos).

Hector Berlioz

Objetivos

- Diseñar parte de un framework¹ con los patrones Método Plantilla, Estado y Command.
 - Introducir las cuestiones de la correspondencia (*mapping*) objeto-relacional (O-R).
 - Implementar la materialización perezosa con Proxies Virtuales.
-

Introducción

La aplicación NuevaEra —como la mayoría— requiere que se almacene y recupere la información en mecanismos de almacenamiento persistente, como una base de datos relacional (BDR). Este capítulo presenta el diseño de un framework para el almacenamiento de objetos persistentes.

Normalmente es mejor conseguir o comprar uno de éstos, ya sea un producto independiente o parte de un contenedor que maneja la persistencia para beans entidad (*entity beans*) si se utiliza EJBs y otras tecnologías de Java. Construir un servicio de persistencia O-R de calidad industrial puede llevar mucho esfuerzo persona-año, y existen cuestiones sutiles que requieren una experiencia especializada. Además, las tecnologías como las que se basan en *Java Data Objects* (JDO) ofrecen soluciones parciales.

Por tanto, la intención no es mostrar un framework industrial o sugerir que se ignoren las tecnologías como JDO, sino más bien utilizar un framework de persistencia como

¹ *N. del T.* Los miembros de la comunidad software de habla hispana utilizan el término original en inglés, aunque a veces se traduce framework por marco, elección que no nos parece muy acertada.

medio para explicar el diseño general de frameworks con patrones, puesto que constituye un caso de estudio especialmente bueno. Es también otro ejemplo de utilización de UML para comunicar un diseño software.

Este framework se presenta para introducir el diseño de los frameworks, no como un enfoque recomendado para el diseño de un servicio de persistencia industrial.

34.1. El problema: objetos persistentes

Asuma que en la aplicación NuevaEra, los datos de la *EspecificacionDelProducto* residen en una base de datos relacional. Estos datos deben traerse a la memoria local durante el uso de la aplicación. Los **objetos persistentes** son aquellos que requieren almacenamiento persistente, como las instancias de *EspecificacionDelProducto*.

Mecanismos de almacenamiento y objetos persistentes

Bases de datos de objetos: Si se utiliza una base de datos de objetos para almacenar y recuperar los objetos, no se necesita ningún servicio de persistencia a medida o de terceras partes adicional. Éste es uno de los diversos atractivos de su uso.

Bases de datos relacionales: Debido al predominio de las BDR, a menudo es necesario que se utilicen, en lugar de las bases de datos de objetos que son más convenientes. Si es éste el caso, surgen algunos problemas debido a la incompatibilidad entre la representación de los datos orientada a registros y la orientada a objetos; estos problemas se estudiarán más adelante. Se requiere un servicio especial para establecer la correspondencia O-R (*mapping O-R*).

Otros: Además de las BDR, a veces se desea almacenar los objetos en otros mecanismos de almacenamiento o formatos, como simples ficheros, estructuras XML, ficheros Palm OS PDB, bases de datos jerárquicas, etcétera. Como con las bases de datos relacionales, existen incompatibilidades entre las representaciones de los objetos y estos formatos que no son orientados a objetos. Y como con las BDR, se requieren servicios especiales que hagan que funcionen con objetos.

34.2. La solución: un servicio de persistencia a partir de un framework de persistencia

Un **framework de persistencia** es un conjunto de tipos de propósito general, reutilizable y extensible, que proporciona funcionalidad para dar soporte a los objetos persistentes. Un **servicio de persistencia** (o subsistema) realmente proporciona el servicio, y se creará con un framework de persistencia. Un servicio de persistencia se escribe normalmente para que trabaje con BDR, en cuyo caso también se conoce como **servicio de correspondencia O-R**. Generalmente, un servicio de persistencia tiene que traducir los objetos a registros (o a alguna otra forma de datos estructurada como XML) y guardarlos en una base de datos, y traducir los registros a objetos cuando los recuperamos de la base de datos.

En cuanto a la arquitectura en capas de la aplicación NuevaEra, un servicio de persistencia es un subsistema dentro de la capa de servicios técnicos.

34.3. Frameworks

Aun a riesgo de simplificar en exceso, un framework es un conjunto *extensible* de objetos para funciones relacionadas. El ejemplo prototípico es un framework de GUI, como las AWT o Swing de Java.

La señal de calidad de un framework es que proporciona una implementación para las funciones básicas e invariables, e incluye un mecanismo que permite al desarrollador conectar las funciones que varían, o extender las funciones.

Por ejemplo, el framework Swing de GUI de Java proporciona muchas clases e interfaces para las funciones principales de la GUI. Los desarrolladores pueden incluir elementos gráficos especializados creando subclases de las clases Swing y redefiniendo ciertos métodos. Los desarrolladores también pueden conectar diversos comportamientos de respuesta a los eventos en las clases de los elementos gráficos predefinidas (como *JButton*) registrando oyentes o suscriptores basados en el patrón Observador. Eso es un framework.

En general, un **framework**:

- Es un conjunto cohesivo de interfaces y clases que colaboran para proporcionar los servicios de la parte central e invariable de un subsistema lógico.
- Contiene clases concretas (y especialmente) abstractas que definen las interfaces a las que ajustarse, interacciones de objetos en las que participar, y otras invariantes.
- Normalmente (aunque no necesariamente) requiere que el usuario del framework defina subclases de las clases del framework existentes para utilizar, adaptar y extender los servicios del framework.
- Tiene clases abstractas que podrían contener tanto métodos abstractos como concretos.
- Confía en el **Principio Hollywood** —“*No nos llame, nosotros le llamaremos*”. Esto significa que las clases definidas por el usuario (por ejemplo, nuevas clases) recibirán mensajes desde las clases predefinidas del framework. Estos mensajes normalmente se manejan implementando métodos abstractos en las superclases.

El siguiente ejemplo del framework de persistencia describirá y explicará estos principios.

Los frameworks son reutilizables

Los frameworks ofrecen un alto grado de reutilización —mucho más que con clases individuales—. En consecuencia, si una organización está interesada (¿y quién no lo está?) en incrementar su grado de reutilización del software, entonces debería enfatizar la creación de frameworks.

34.4. Requisitos para el servicio y framework de persistencia

Para la aplicación de PDV NuevaEra, necesitamos un servicio de persistencia que se construya con un framework de persistencia (que se podría utilizar también para crear otros servicios de persistencia). Llamemos al framework FWP (Framework de Persis-

tencia). FWP es un framework simplificado —un framework de persistencia desarrollado de calidad industrial queda fuera del alcance de esta introducción—.

El framework debería proporcionar funciones para:

- almacenar y recuperar los objetos en un mecanismo de almacenamiento persistente
- *confirmar y deshacer (commit y rollback)* las transacciones

El diseño debe ser extensible para dar soporte a diferentes mecanismos de almacenamiento, como BDRs, registros en ficheros simples, o XML en ficheros.

34.5. Ideas claves

Las siguientes ideas claves se estudiarán en las secciones que vienen a continuación:

- **Correspondencia:** Se debe establecer alguna correspondencia (*mapping*) entre una clase y su almacenamiento persistente (por ejemplo, una tabla en una base de datos), y entre los atributos de los objetos y los campos (columnas) en un registro. Es decir, debe existir un **correspondencia de esquemas** entre los dos esquemas.
- **Identidad de objeto:** Los registros y los objetos tienen un único identificador de objeto para relacionar fácilmente los registros con los objetos, y asegurar que no hay duplicados inapropiados.
- **Conversor de base de datos:** Una Fabricación Pura conversor (*mapper*) de base de datos es responsable de la materialización y desmaterialización.
- **Materialización y desmaterialización:** La materialización es el acto de transformar una representación de datos no orientada a objetos (por ejemplo, registros) de un almacenamiento persistente en objetos. La desmaterialización es la actividad opuesta (también conocida como *passivation*).
- **Caché:** Los servicios persistentes almacenan en una caché los objetos materializados por razones de rendimiento.
- **Estado de transacción de los objetos:** Es útil conocer el estado de los objetos en función de sus relaciones con la transacción actual. Por ejemplo, es útil conocer qué objetos se han modificado (están *sucios*) de manera que es posible determinar si es necesario que se guarden de nuevo en su almacenamiento persistente.
- **Operaciones de transacción:** Operaciones confirmar y deshacer (*commit y rollback*).
- **Materialización perezosa:** No todos los objetos se materializan de una vez; una instancia particular sólo se materializa bajo demanda, cuando se necesita.
- **Proxies virtuales:** La materialización perezosa se puede implementar utilizando una referencia inteligente que se conoce como proxy virtual.

34.6. Patrón: Representación de Objetos como Tablas

¿Cómo conviertes un objeto en un registro o esquema de base de datos relacional?

El patrón **Representación de Objetos como Tablas** [BW96] propone la definición de una tabla en una BDR por cada clase de objeto persistente. Los atributos de los

objetos que contienen tipos de datos primitivos (número, cadena de texto, booleano, etcétera) se corresponden con las columnas.

Si un objeto sólo tiene atributos de tipos de datos primitivos, la correspondencia es directa. Pero como veremos, las cosas no son tan simples puesto que los objetos podrían tener atributos que hacen referencia a otros objetos complejos, mientras que el modelo relacional requiere que los valores sean atómicos (esto es, la Primera Forma Normal) (ver Figura 34.1).

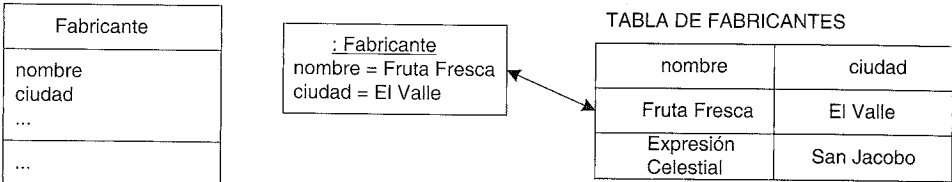


Figura 34.1. Correspondencia entre objetos y tablas.

34.7. Perfil (Profile) de modelado de datos en UML

A propósito de las BDR, no es sorprendente que UML se haya convertido en una notación muy utilizada para los **modelos de datos**. Fíjese que uno de los artefactos oficiales del UP es el Modelo de Datos, que forma parte de la disciplina de Diseño. La Figura 34.2 ilustra alguna notación UML para el modelado de datos.

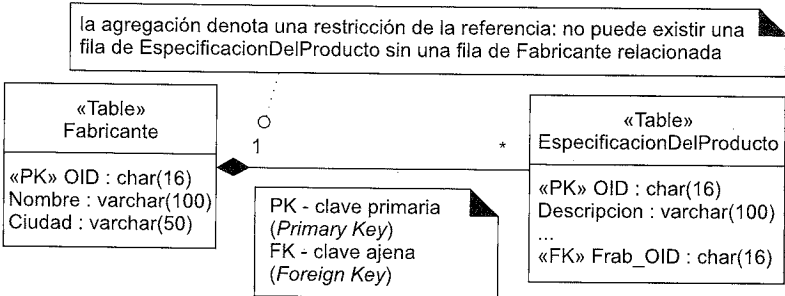


Figura 34.2. Ejemplo del Perfil (Profile) de Modelado de Datos en UML.

Estos estereotipos no forman parte del núcleo de UML —son extensiones—. Generalizando, UML tiene el concepto de **perfil** (perfil de UML): un conjunto coherente de estereotipos de UML, valores etiquetados y restricciones para un propósito específico. La Figura 34.2 ilustra parte del Perfil de Modelado de Datos de UML propuesto (al OMG); en el momento en el que se escribió este libro no se había aprobado. Un perfil no tiene que estar aprobado por el OMG para ser un perfil, aunque se están enviando algunos de uso muy extendido —como el modelado de datos— para que se aprueben.

34.8. Patrón: Identificador de Objeto

Es conveniente contar con una forma consistente de relacionar los objetos con los registros, y ser capaces de asegurar que la materialización repetida de un registro no da como resultado objetos duplicados.

El patrón **Identificador de Objeto** [BW96] propone asignar un **identificador de objeto** (OID) a cada registro y objeto (o proxy de un objeto).

Un OID normalmente es un valor alfanumérico; cada uno es único para un objeto específico. Existen varios enfoques para generar identificadores únicos para los OIDs, variando desde únicos para una base de datos, a únicos globalmente: generadores de secuencia de bases de datos, la estrategia de generación de claves Alto-Bajo [Ambler00], y otros.

En el campo de los objetos, un OID se representa mediante una interfaz o clase OID que encapsula el valor real y su representación. En un BDR, normalmente se almacena como un valor de tipo carácter de longitud fija.

Cada tabla tendrá un OID como clave primaria, y cada objeto también tendrá (directa o indirectamente) un OID. Si se asocia cada objeto con un OID, y cada tabla tiene un OID como clave primaria, cada objeto se corresponde de manera única con una fila de alguna tabla (ver Figura 34.3).

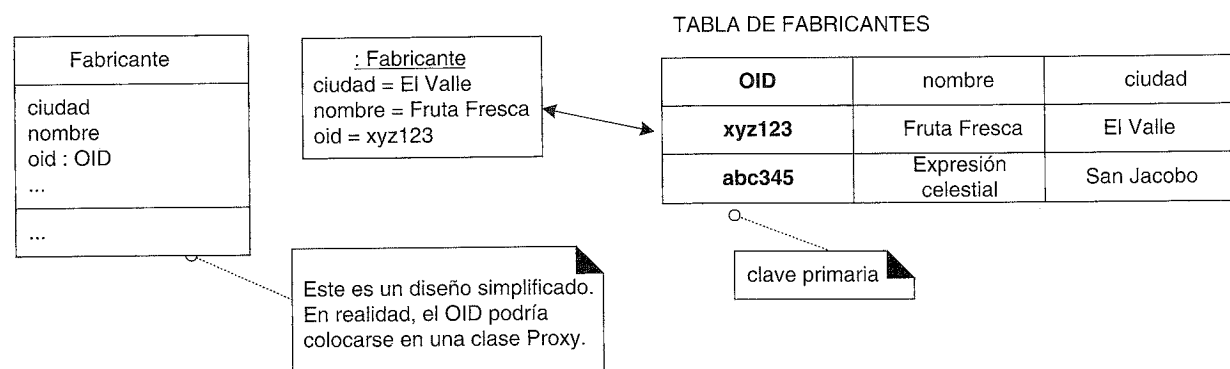


Figura 34.3. Los identificadores de los objetos enlazan objetos y registros.

Ésta es una vista simplificada del diseño. En realidad, el OID podría no colocarse exactamente en el objeto persistente —aunque sea posible—. En lugar de eso, se podría colocar en un objeto Proxy que envuelve al objeto persistente. En el diseño influye la elección del lenguaje.

Un OID también proporciona un tipo de clave consistente para utilizarla en la interfaz con el servicio de persistencia.

34.9. Acceso al servicio de persistencia con una Fachada

El primer paso del diseño de este subsistema es definir una fachada para estos servicios; recordemos que la Fachada es un patrón común para proporcionar una interfaz uniforme a un subsistema. Para empezar, se necesita una operación para recuperar un objeto dado un OID. Pero además del OID, el subsistema necesita conocer el tipo del objeto que se va a materializar; por tanto, también se debe proporcionar el tipo de la clase. La Figura 34.4 ilustra algunas operaciones de la fachada y su uso en colaboración con uno de los adaptadores de servicios de NuevaEra.

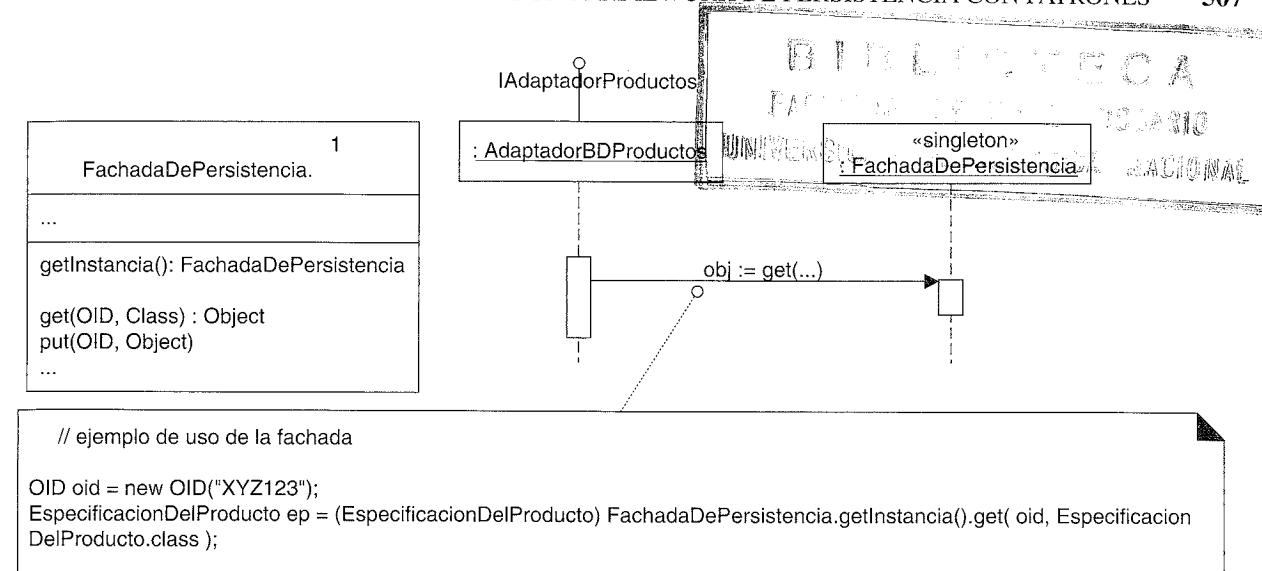


Figura 34.4. La FachadaDePersistencia.

34.10. Correspondencia de los objetos: patrón Conversor (Mapper) de Base de Datos o Intermediario (Broker) de Base de Datos

La *FachadaDePersistencia* —como se cumple en todas las fachadas— no hace ella misma el trabajo, sino que delega las peticiones en objetos del subsistema.

¿Quién debe ser el responsable de la materialización y desmaterialización de los objetos (por ejemplo, una *EspecificacionDelProducto*) procedentes de un almacenamiento persistente?

El patrón Experto en Información sugiere que la propia clase del objeto (*EspecificacionDelProducto*) persistente es candidata, porque tiene algunos de los datos (los datos que se van a almacenar) que necesita la responsabilidad.

Si una clase de objetos persistentes define el código para almacenarse ella misma en una base de datos, se denomina diseño de **correspondencia directa**. Se puede utilizar la correspondencia directa si el código relacionado de la base de datos se genera y se inyecta automáticamente en la clase mediante un compilador de post-procesamiento, y el desarrollador nunca tiene que ver o mantener este código de base de datos complejo que añade confusión a su clase.

Pero si la correspondencia directa se añade y mantiene de manera manual, tiene varios defectos y tiende a no ser escalable en cuanto a la programación y el mantenimiento. Entre los problemas encontramos:

- Fuerte acoplamiento de la clase de objetos persistentes y el conocimiento del almacenamiento persistente —violación de Bajo Acoplamiento—.
- Responsabilidades complejas en un área nueva y no relacionada con las responsabilidades previas del objeto —violación de Alta Cohesión y mantenimiento de la

separación de intereses—. Cuestiones relacionadas con servicios técnicos se mezclan con otras propias de la lógica de la aplicación.

Estudiaremos un enfoque clásico de **correspondencia indirecta**, que utiliza otros objetos para establecer la correspondencia con los objetos persistentes.

Parte de este enfoque es utilizar el patrón **Intermediario (Broker) de Base de Datos** [BW95]. Éste propone crear una clase que sea responsable de materializar y desmaterializar un objeto almacenado. También se le ha llamado patrón **Conversor (Mapper) de Base de Datos** en [Fowler01], que es un nombre más adecuado que Broker de Bases de Datos, puesto que describe su responsabilidad, y el término “broker” en el diseño de los sistemas distribuidos [BMRSS96] tiene un significado distinto, que existe desde hace mucho tiempo².

Se define una clase diferente que establece la correspondencia para cada clase de los objetos persistentes. La Figura 34.5 ilustra que cada objeto persistente podría tener su propia clase que lleve a cabo la correspondencia, y que podrían existir diferentes tipos de conversores para diferentes tipos de mecanismos de almacenamiento. Un extracto del código:

```
class FachadaDePersistencia
{
    //...
    public Object get(OID oid, Class clasePersistente)
    {
        //la clave de IConversor es la Clase del objeto persistente
        IConversor conversor = (IConversor) conversores.get(clasePersistente);

        //delega
        return conversor.get(oid);
    }
    //...
}
```

Aunque este diagrama señala dos conversores para la *EspecificacionDelProducto*, sólo uno de ellos estará activo en un servicio de persistencia en funcionamiento.

Conversores basados en metadatos

Un diseño de conversores más flexible, pero más difícil, se basa en **metadatos** (datos sobre los datos). A diferencia de las clases hechas a mano que establecen la correspondencia de manera individual para diferentes tipos persistentes, los conversores basados en metadatos generan dinámicamente la correspondencia entre un esquema de objetos y otro esquema (como el relacional) en base a la lectura de metadatos que describen la correspondencia, como “La TablaX se corresponde con la Clase Y; la columna Z se corresponde con la propiedad P del objeto” (llega a ser mucho más complejo). Este enfoque es viable en lenguajes con capacidades de programación reflexiva, como Java, C# o Smalltalk, y resulta difícil para aquellos que no las tienen, como C++.

Estableciendo la correspondencia en base a metadatos, podemos cambiar la correspondencia de esquemas en un almacenamiento externo y tendrá efecto en el sistema en marcha, sin que haya que cambiar el código fuente —Variaciones Protegidas con respecto a las variaciones en los esquemas—.

No obstante, una cualidad útil del framework que se ha presentado aquí es que se pueden utilizar los conversores escritos a mano o basados en metadatos, sin afectar a los clientes —encapsulación de la implementación—.

34.11. Diseño del framework con el patrón Método Plantilla

La siguiente sección describe algunas de las características esenciales del diseño de los Conversores de Bases de Datos, que constituyen una parte central del FWP. Estas características de diseño se basan en el patrón de diseño GoF **Método Plantilla (Template Method)** [GHJV95]³. Este patrón es una parte esencial del diseño del framework⁴, y es familiar para la mayoría de los programadores OO por la práctica si no por el nombre.

³ Este patrón no está relacionado con las plantillas (template) de C++. El patrón describe la *plantilla* de un algoritmo.
⁴ De manera más específica, de los **frameworks de caja blanca**. Normalmente éstos son frameworks orientados a la definición de subclases y jerarquías de clases que requieren que los usuarios conozcan algo acerca de su diseño y estructura; de ahí lo de caja blanca.

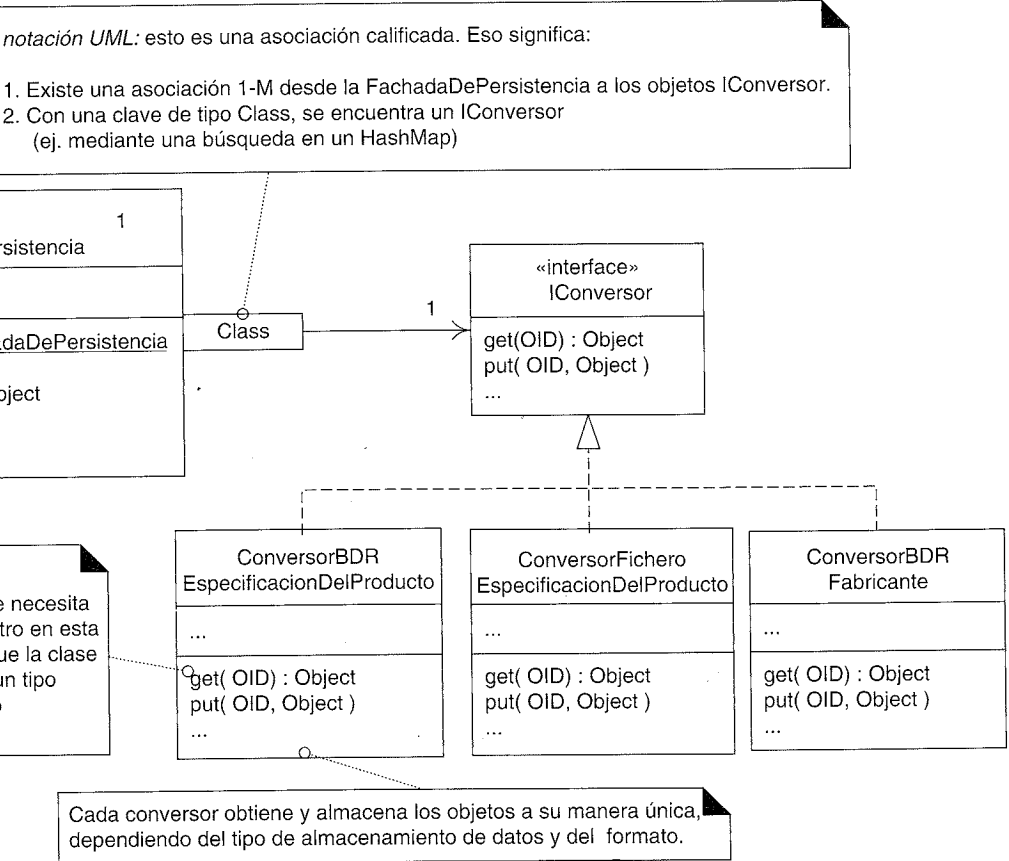


Figura 34.5. Conversores de bases de datos.

² En los sistemas distribuidos, un **broker** es un proceso del servidor front-end que delega las tareas en los procesos del servidor back-end.